

Automated Electricity Demand Forecasting and Intelligent Model Selection Platform



Author: Paras Rimal
Student ID: 25123819
Lecturer: Umesh Regmi
Moodle Title: Django and Fast API

Abstract

Accurate day-ahead forecasts of electricity demand support the planning of generation and purchases, but a forecasting model is only useful if it is trustworthy, monitored and replaced only when something better is proven. This report describes GridPredict, an end-to-end system that forecasts the hourly electric load of Ahmedabad, India, 24 hours ahead using one year of hourly weather and load data (8,779 hours, November 2021 to November 2022).

The report documents the data analysis, the cleaning rules (24 invalid irradiance values and 79 load dropouts set to missing, never interpolated), and a leakage-safe feature pipeline with calendar, weather, lag (24, 48, 168 hours) and rolling features, giving 8,532 modelling rows. Random Forest, XGBoost and LightGBM are compared against naive baselines on a chronological split and by a seven-fold rolling-origin backtest. On validation, XGBoost beat the "same hour yesterday" baseline (MAE 8.47 against 9.25 MW), and a Random Forest without season columns won the backtest (mean MAE 8.44 MW, 6 of 7 folds). However, on the held-out test period both lost to the baseline (10.67 and 10.40 MW against 9.01 MW). An error analysis shows that the Random Forest is slightly better in hours up to 100 MW but much worse in high-load hours, which is consistent with the failure of tree ensembles to follow a rising load level.

A promotion gate requiring an improvement of more than 0.3 MW rejected both challengers, so the naive baseline remains in production. A drift check flags 14 of 19 features (73.7 %), mainly because of seasonal change, while the forecast error is unchanged. The system is served by FastAPI with a fail-open Redis cache, a Django REST backend, a daily Celery monitoring job and a React dashboard, packaged in six Docker services and covered by 187 automated test functions. The main contribution is a trustworthy process that detects and reports a negative result rather than deploying a worse model.

Keywords: short-term load forecasting; Random Forest; XGBoost; LightGBM; rolling-origin backtest; data leakage; model registry; drift monitoring; FastAPI; Django; React

Contents

1. Introduction.....	7
1.1 Background and motivation.....	7
1.2 Problem statement.....	7
1.3 Aim and objectives.....	7
1.4 Scope and main findings.....	8
1.5 Contributions.....	8
1.6 Structure of the report.....	8
2. Literature Review and Theoretical Background.....	9
2.1 Short-term load forecasting.....	9
2.2 Naive baselines.....	9
2.3 Tree-based learning methods.....	9
2.4 Evaluation metrics.....	10
2.5 Validation of time-series models and data leakage.....	10
2.6 Concept drift and monitoring.....	10
2.7 Governance of machine learning systems.....	11
3. System Requirements and Architecture.....	12
3.1 Functional requirements.....	12
3.2 Technology stack.....	12
3.3 Overall architecture.....	12
3.4 Repository structure.....	13
3.5 Design principles.....	14
4. Data Description and Exploratory Analysis.....	15
4.1 Candidate datasets and the choice of the primary dataset.....	15
4.2 The Ahmedabad dataset.....	15
4.3 Data quality and cleaning rules.....	15
4.4 Daily, weekly and seasonal patterns.....	16
4.5 Relationship with the weather.....	17
4.6 Autocorrelation.....	18
4.7 Level differences between the periods.....	18
5. Data Preprocessing and Feature Engineering.....	20
5.1 Configuration (config.py).....	20
5.2 Loading, timeline check and cleaning (data_loader.py).....	20
5.3 Calendar features (calendar_features.py).....	21
5.4 Lag features (lag_features.py).....	21
5.5 Rolling features (rolling_features.py).....	21
5.6 Assembling the feature table (pipeline.py).....	22
6. Modelling Methodology.....	24
6.1 Chronological split (split.py).....	24
6.2 Baselines (baseline.py).....	24
6.3 Models and settings (model_factory.py).....	24
6.4 Training and scoring (trainer.py and metrics.py).....	25
6.5 Comparison and the first selection rule (compare.py and selection.py).....	25
6.6 Rolling-origin backtest (backtest.py).....	25
6.7 The second selection rule (backtest_selection.py).....	26
6.8 Training runs and artifacts (train_pipeline.py, train_candidate.py, artifacts.py).....	26
6.9 Registry and promotion gate (registry.py).....	27

7. Experiments and Results.....	28
7.1 Baselines and the validation comparison.....	28
7.2 The single-window selection fails on the test set.....	28
7.3 Rolling backtest and re-selection.....	29
7.4 The backtest-selected candidate on the test set.....	31
7.5 Feature importance.....	33
7.6 The promotion gate in action.....	34
7.7 Summary of the results.....	35
8. Monitoring and Retraining.....	36
8.1 Design.....	36
8.2 Data drift (drift.py).....	36
8.3 Performance degradation (performance.py).....	37
8.4 The combined decision (retraining.py).....	37
8.5 Results of the monitoring run.....	37
8.6 Scheduling with Celery (monitoring/tasks.py and migration 0002).....	39
9. Serving Layer, Backend and Dashboard.....	40
9.1 Production model wrapper (inference/production_model.py).....	40
9.2 FastAPI ML service (main.py, routes/predict.py, schemas/predict.py).....	40
9.3 Prediction cache (caching/prediction_cache.py).....	40
9.4 Django REST backend.....	41
9.5 React dashboard.....	42
9.6 Containerised deployment.....	42
10. Testing and Quality Assurance.....	44
10.1 Tests that protect the most important guarantees.....	45
11. Discussion.....	47
11.1 What the results mean.....	47
11.2 The value of honest evaluation.....	47
11.3 Strengths.....	47
11.4 Limitations and threats to validity.....	47
11.5 Recommendations.....	48
12. Conclusions and Future Work.....	49
References.....	50
Appendix A. Source File Inventory.....	52
Appendix B. Central Configuration (config.py).....	54
Appendix C. Core Source Code Listings.....	57
Appendix D. Model Metadata.....	72
Appendix E. Model Registry (models/registry.json).....	76
Appendix F. Reproduction Instructions.....	77
Appendix G. API Example (illustrative).....	78
Appendix H. Complete Test Inventory.....	79
Appendix I. Environment and Pinned Versions.....	82

(In Microsoft Word, if the page numbers are not shown, right-click the table and choose Update Field.)

List of Figures

- Figure 1** GridPredict system architecture: React front end, Django REST API, FastAPI ML service, Redis, Celery and the file-based data and model stores.
- Figure 2** Hourly electric load, daily mean and daily mean temperature of Ahmedabad (November 2021 to November 2022); orange markers show the meter dropout hours.
- Figure 3** Left: the irradiance sensor code –999 over 24 hours. Right: lengths of the 22 load dropout runs (79 hours in total).
- Figure 4** Mean load by hour of day (band = ± 1 standard deviation), by day of week and by month.
- Figure 5** Pearson correlation matrix of the weather variables and the load (cleaned data).
- Figure 6** Autocorrelation function of the hourly load for lags up to 336 hours; the three lags used as features are marked.
- Figure 7** Evidence for rejecting PowerLoad_Dataset.csv: flat hourly profile, uniform-like temperature histogram and near-zero autocorrelation.
- Figure 8** Left: load against the load of the same hour yesterday (dashed line = perfect repetition). Right: distribution of the load in each chronological split.
- Figure 9** End-to-end ML pipeline of GridPredict, from the raw CSV to the production API.
- Figure 10** Leakage guard: the forecast is made at $t - 24$ h, so every lag (orange dots) and rolling window ends at or before that moment.
- Figure 11** Chronological train, validation and test split of the hourly load.
- Figure 12** Selection logic: validation rule, backtest rule, single look at the test set and the promotion gate.
- Figure 13** Validation MAE and R^2 of the baseline and the three models.
- Figure 14** Test-set MAE, RMSE and R^2 of the baseline, XGBoost (selected on validation) and the Random Forest candidate (selected by backtest).
- Figure 15** Rolling backtest: number of folds in which each model beats the baseline (left) and mean MAE (right).
- Figure 16** Left: per-fold MAE of the baseline and the two Random Forest variants. Right: per-fold bias of the Random Forest without season columns.
- Figure 17** Test period: a two-week zoom (top) and the daily means over the whole test period (bottom) for the actual load, the Random Forest and the baseline.
- Figure 18** Left: distribution of forecast errors. Centre: predicted against actual load for the Random Forest. Right: test MAE by hour of day.
- Figure 19** Impurity-based feature importance of the Random Forest candidate (no season columns).
- Figure 20** Promotion gate: MAE improvement of each challenger over the production baseline compared with the required margin of +0.30 MW.
- Figure 21** Retraining decision: the drift check and the performance check are combined with a logical OR.
- Figure 22** Stored monitoring result: the drift share exceeds its threshold, the performance change is negligible.
- Figure 23** Per-feature drift distance between the training period (reference) and the test period (current), recomputed with SciPy; red bars exceed the 0.1 threshold.
- Figure 24** Life cycle of a POST /predict request, including the cache hit path and the fail-open behaviour.
- Figure 25** The eight pages of the dashboard and the back-end endpoints that feed each of them.
- Figure 26** Number of test functions per module (blue: ML package, orange: Django apps).

List of Tables

- Table 1**Principal technologies used in the project (versions from requirements.txt, package.json and the Dockerfiles)
- Table 2**Repository layout and size
- Table 3**Variables in the Ahmedabad dataset (summary statistics are from the raw file, before cleaning)
- Table 4**Data quality issues and the rule applied
- Table 5**The 19 model input features (the Random Forest candidate uses the 16 features that do not carry a † mark)
- Table 6**Chronological split of the 8,532 modelling rows
- Table 7**Model settings from config.py
- Table 8**Validation scores (July to August 2022; baseline scored on 1,448 rows, 20 rows skipped)
- Table 9**Test-set scores of the baseline and the XGBoost model selected on validation (baseline: 1,474 rows, 7 skipped)
- Table 10**Rolling backtest summary (7 monthly folds, documented project results)
- Table 11**Recomputed per-fold backtest (Random Forest; MAE and bias in MW; bias = mean of prediction minus actual)
- Table 12**Test-set scores of the Random Forest candidate (no season columns)
- Table 13**Test-set error by load level (recomputed from the saved model; rows where the baseline can predict)
- Table 14**Top feature importances of the saved Random Forest model
- Table 15**Promotion history in models/registry.json (decisions recorded on 25 September 2026)
- Table 16**Summary of all model evaluations (MAE in MW)
- Table 17**Drift flags by feature group (SciPy recomputation; 14 of 19 features exceed 0.1)
- Table 18**Endpoints of the FastAPI ML service (port 8000)
- Table 19**Endpoints of the Django backend (port 8001)
- Table 20**Services in docker-compose.yml
- Table 21**Test functions by area
- Table 22**Modules of ml_service/app
- Table 23**Inspection scripts in the notebooks folder
- Table 24**Test modules and functions
- Table 25**Selected pinned dependencies (requirements.txt)

List of Abbreviations

Table 26: Abbreviations used in this report

Abbreviation	Meaning
API	Application programming interface
CORS	Cross-origin resource sharing
DRF	Django REST framework
GJ	Gujarat (holiday subdivision code)
JS distance	Jensen-Shannon distance
MAE	Mean absolute error
MAPE	Mean absolute percentage error
MW	Megawatt
NaN	Not a number (missing value)
R^2	Coefficient of determination
RMSE	Root mean squared error
TTL	Time to live (cache expiry)
XGBoost	Extreme gradient boosting
LightGBM	Light gradient boosting machine

1. Introduction

1.1 Background and motivation

Electricity cannot be stored economically at grid scale, so generation has to match consumption almost instantly. System operators therefore plan generation, purchases and reserves ahead of time, and the quality of those plans depends on how well the future demand (the "load") is forecast. A day-ahead forecast, which predicts the load for every hour of tomorrow, is a standard planning horizon in grid operations. Short-term load forecasting has been studied for decades, and reviews such as Hippert, Pedreira and Souza (2001) and Hong and Fan (2016) show that load is shaped by a small number of recurring drivers: the time of day, the day of the week, the season, public holidays and, above all, the weather.

This project, named **GridPredict** (repository: [GridDemand AI](#)), builds a complete day-ahead forecasting system for the hourly electric load of Ahmedabad, India. It is not only a model. It is an end-to-end machine learning system that covers data cleaning, leakage-safe feature engineering, model comparison, rolling-origin backtesting, a model registry with a promotion gate, drift and performance monitoring, a prediction service with caching, a web API, a scheduled monitoring job and a React dashboard. Sculley et al. (2015) argue that in real machine learning systems the model code is only a small part of the whole, and most of the engineering risk lies in the surrounding infrastructure. GridPredict is designed with that argument in mind.

1.2 Problem statement

Given the hourly weather and load history of Ahmedabad from November 2021 to November 2022, the system must forecast the electric load (in megawatts, MW) of a target hour 24 hours in advance, decide automatically and honestly whether a trained model is better than a simple rule, and keep watching the incoming data so that a model is retrained only when there is statistical evidence that it should be.

1.3 Aim and objectives

The aim is to design, implement and evaluate a reproducible, production-style day-ahead load forecasting system. The objectives are:

1. to select and clean a suitable dataset and document every cleaning decision;
2. to engineer calendar, weather, lag and rolling features that cannot leak information from the future;
3. to establish naive baselines and compare Random Forest, XGBoost and LightGBM against them on a chronological split;
4. to build a rolling-origin backtest and a documented selection rule that is more reliable than a single validation window;

5. to evaluate the selected candidate once on a held-out test period and to report the result honestly, whether or not it is favourable;
6. to implement a model registry and a promotion gate so that a model replaces production only when it is clearly better;
7. to implement drift and performance monitoring and a retraining decision that is triggered by evidence rather than by the mere arrival of new data; and
8. to serve the production model through a FastAPI service and a Django REST backend with a React dashboard, packaged in Docker.

1.4 Scope and main findings

The system is built and tested on a single dataset that covers one calendar year, which limits what can be concluded about long-term behaviour. The principal findings are summarised below and are developed in Chapters 7 and 8.

- The "same hour yesterday" baseline is strong: on the validation period its mean absolute error (MAE) is 9.25 MW, and on the test period it is 9.01 MW.
- On validation, all three machine learning models beat the baseline (best MAE 8.47 MW, XGBoost). A single-window rule therefore selected XGBoost, but on the held-out test period XGBoost scored 10.67 MW and lost to the baseline.
- A seven-fold rolling backtest chose a Random Forest without season columns (mean MAE 8.44 MW, 6 of 7 folds better than the baseline). That candidate also lost on the test period (10.40 MW against 9.01 MW).
- The promotion gate correctly rejected both challengers, and the naive baseline remains in production. The monitoring job flagged data drift in 14 of 19 features (73.7 %), which is expected because the test period (September to November) differs from the training period (November to June).

1.5 Contributions

The contribution of the work is methodological rather than a record-breaking accuracy figure. The project demonstrates (i) a leakage-safe feature pipeline whose guarantees are enforced by assertions and by end-to-end tests, (ii) a selection procedure that was fixed before the comparison was run, (iii) an honest negative result that is reported instead of hidden, and (iv) an automated governance layer (promotion gate and retraining rule) that prevented a worse model from reaching production.

1.6 Structure of the report

Chapter 2 reviews the theory and the literature. Chapter 3 describes the architecture and the repository. Chapter 4 analyses the data. Chapter 5 explains the preprocessing and feature engineering code. Chapter 6 describes the modelling methodology. Chapter 7 presents the experiments and results. Chapter 8 covers monitoring and retraining. Chapter 9 describes the

serving layer, backend and dashboard. Chapter 10 covers testing. Chapter 11 discusses the results and limitations, and Chapter 12 concludes. The references follow in Harvard style, and the appendices contain configuration tables, source code listings, model metadata and reproduction instructions.

2. Literature Review and Theoretical Background

2.1 Short-term load forecasting

Load forecasting is usually divided by horizon into very short-term, short-term (hours to a few weeks), medium-term and long-term problems. Short-term forecasting is the most common operational task and is the focus of the tutorial review by Hong and Fan (2016), who stress the importance of calendar effects, temperature and the choice of evaluation scheme. Earlier work reviewed by Hippert, Pedreira and Souza (2001) concentrated on neural networks, and noted that many reported results were difficult to compare because of weak validation practice. This observation is directly relevant to the present project: much of its effort goes into validating honestly rather than into tuning models.

Load has three characteristic regularities. It has a **daily cycle** (low at night, high in the afternoon), a **weekly cycle** (weekdays differ from weekends), and a **seasonal cycle** (cooling and heating demand follow the weather). These cycles are visible in the Ahmedabad data in Chapter 4. Because the cycles repeat, the load observed 24 hours or 168 hours earlier is usually informative about the load now, which motivates both the naive baselines and the lag features used in this project.

2.2 Naive baselines

Hyndman and Athanasopoulos (2021) describe the seasonal naive method, which forecasts a value equal to the last observation from the same season. With hourly data and a daily season, the seasonal naive forecast is "the same hour yesterday". Such methods are cheap, transparent and often hard to beat, so they are the correct minimum standard for any learned model. GridPredict uses the 24-hour and 168-hour naive rules as baselines and makes the 24-hour rule the default production model until a challenger proves better.

2.3 Tree-based learning methods

2.3.1 Random Forest

A Random Forest (Breiman, 2001) is an ensemble of decision trees. Each tree is trained on a bootstrap sample of the rows, and at each split only a random subset of the features is considered, which decorates the trees. The forest prediction for regression is the average of the tree predictions. In the project the forest has 300 trees, requires at least 2 rows in every leaf and considers half of the features at each split. A property that matters later in this report

is that a tree predicts the average of the training targets that fall in a leaf, so the forest output is a piecewise-constant function that stays within the range of training targets and is smoothed towards typical historical values (Hastie, Tibshirani and Friedman, 2009). Such a model does not extrapolate a trend or a level shift that was not present in its training data.

2.3.2 Gradient boosting: XGBoost and LightGBM

Gradient boosting (Friedman, 2001) builds trees sequentially, each new tree fitting the errors of the ensemble so far. XGBoost (Chen and Guestrin, 2016) is a regularised and highly optimised implementation, and LightGBM (Ke et al., 2017) uses histogram-based, leaf-wise tree growth for speed. Both are strong on tabular data. In this project they are used with fixed, deliberately untuned settings (300 trees, learning rate 0.05, row and column subsampling of 0.8) so that the comparison does not depend on how much tuning effort each model received.

2.4 Evaluation metrics

Four metrics are reported. Let y be the actual load, $\hat{y}$ the forecast and n the number of hours scored.

$$\begin{aligned} MAE &= (1/n) \sum |y - \hat{y}| & RMSE &= \sqrt{[(1/n) \sum (y - \hat{y})^2]} \\ MAPE &= (100/n) \sum |(y - \hat{y}) / y| & R^2 &= 1 - \sum (y - \hat{y})^2 / \sum (y - \bar{y})^2 \end{aligned}$$

The MAE is in megawatts and is easy to interpret; Willmott and Matsuura (2005) argue that it is a clearer measure of average error than the RMSE, while Chai and Draxler (2014) show that the RMSE is appropriate when large errors are especially costly. The project reports both. The MAPE expresses the error as a percentage of the actual load, but it is undefined at zero and can be distorted when the actual value is small (Hyndman and Koehler, 2006); the code therefore raises an error if an actual value is exactly zero. R^2 compares the model with the trivial forecast of the mean. The selection rules in this project are based on the MAE.

2.5 Validation of time-series models and data leakage

Ordinary random cross-validation is not suitable for time series because it lets the model train on the future and test on the past. Tashman (2000) reviews out-of-sample testing for forecasting and recommends rolling-origin evaluation, in which the forecast origin moves forward through time. Bergmeir and Benítez (2012) discuss when cross-validation can be used for time series predictors and when it cannot. GridPredict therefore uses a chronological split and an expanding-window rolling-origin backtest, with a gap equal to the forecast horizon between the training data and each test fold.

Data leakage means that information that would not be available at prediction time finds its way into the training features (Kaufman et al., 2012). In a day-ahead forecast the forecast is issued 24 hours before the target hour, so the newest load value that is known is the load at the target hour minus 24 hours. Any lag or rolling window that touches a later hour leaks the future. The project prevents this in three layers: an assertion in the configuration file,

validation inside the feature functions, and end-to-end tests that deliberately corrupt the "future" 23 hours and verify that the features do not change.

2.6 Concept drift and monitoring

A deployed model can lose accuracy because the world changes. Gama et al. (2014) survey concept drift and the methods used to detect and adapt to it. In practice, two kinds of signals are monitored: changes in the distribution of the input features (data drift) and changes in the forecast error (performance degradation). The project uses the Evidently library to compare feature distributions. For numeric columns the drift score is a normalised Wasserstein distance, which measures how much "earth" must be moved to turn one distribution into the other (Vallender, 1974), and for categorical or binary columns it is the Jensen-Shannon distance (Lin, 1991); both are compared with a threshold of 0.1. On top of the per-column flags the project requires that more than half of the columns drift before the dataset as a whole is considered to have drifted, so that a few unstable columns do not trigger a retrain.

2.7 Governance of machine learning systems

Sculley et al. (2015) describe the hidden technical debt that accumulates in machine learning systems, including entanglement, undeclared consumers and unmonitored changes. Two ideas from this literature are implemented in GridPredict. First, every model is stored as an immutable, timestamped artifact with a metadata file, and a registry file records which model is in production and the full history of promotion decisions. Second, a challenger replaces the production model only if it beats it by a stated margin, which is a guard against swapping models because of differences that are the size of the noise.

3. System Requirements and Architecture

3.1 Functional requirements

- Load, validate and clean the raw hourly data without editing the raw file.
- Build leakage-safe features for a 24-hour horizon.
- Train and compare Random Forest, XGBoost and LightGBM against naive baselines.
- Select a model with a rule that is documented and fixed in advance.
- Save each trained model with metadata, and never overwrite an earlier model.
- Decide whether a challenger may replace the production model, and record every decision.
- Detect data drift and performance degradation and recommend retraining only on evidence.
- Serve single and batch predictions through an HTTP API, with caching.

- Expose the history, monitoring results and feature importance to a web dashboard.

3.2 Technology stack

Table 1: Principal technologies used in the project (versions from requirements.txt, package.json and the Dockerfiles)

Layer	Technology	Version / role
Language	Python	3.12 (Docker image python:3.12-slim)
Data	pandas, NumPy, SciPy	pandas 3.0.6, numpy 2.5.3, scipy 1.18.1
Machine learning	scikit-learn, XGBoost, LightGBM	1.9.1, 3.4.1, 4.7.0; models and metrics
Monitoring	Evidently	0.7.23; DataDriftPreset
Calendar	holidays	0.105; holidays of India, subdivision GJ (Gujarat)
ML service	FastAPI, Uvicorn, Pydantic	0.141.1, 0.53.0, 2.13.5; prediction API
Backend	Django, Django REST framework	6.0.8, 3.18.1; REST API and SQLite database
Scheduling	Celery, django-celery-beat, Redis	5.6.3, 2.9.0, Redis 7; daily monitoring task
Cache	Redis (redis-py 8.1.0)	prediction cache, TTL 300 s
Frontend	React, Vite, Tailwind CSS, Recharts	React 19.2, Vite 8, Tailwind 4.3, Recharts 3.10
Deployment	Docker and docker-compose	6 services: redis, fastapi, django, celery_worker, celery_beat, frontend
Testing	pytest	9.1.1; 187 test functions

3.3 Overall architecture

GridPredict is organised as a set of cooperating services (Figure 1). The React single-page application calls two back ends. The FastAPI ML service holds the production model and answers prediction, health, model and metrics requests. The Django REST service stores forecasts, model versions and monitoring records in SQLite and also exposes the historical demand, feature importance and modelling findings to the dashboard. Redis has two roles: it is the message broker for Celery and the cache for predictions. A Celery worker and a Celery beat scheduler run the daily drift and performance check. The ML code lives in one Python package, `ml_service`, which the Django project imports directly, so the training, registry and monitoring logic exists in exactly one place.

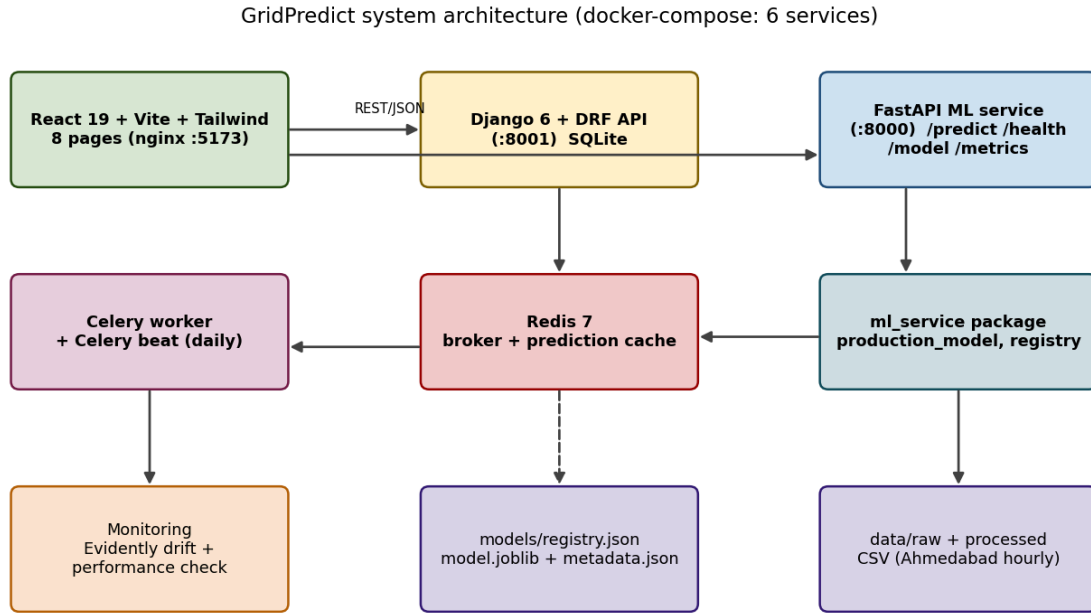


Figure 1: GridPredict system architecture: React front end, Django REST API, FastAPI ML service, Redis, Celery and the file-based data and model stores.

3.4 Repository structure

The repository is organised by responsibility. The ML pipeline is divided into small modules, each with one purpose, which is also how the test suite is organised (Chapter 10).

Table 2: Repository layout and size

Path	Content	Notes
ml_service/app/preprocessing	data_loader, calendar_features, lag_features, rolling_features, pipeline	Raw CSV to model-ready table
ml_service/app/training	split, baseline, model_factory, trainer, compare, backtest, train_pipeline, train_candidate, artifacts, registry	Training, backtesting, saving, promotion
ml_service/app/evaluation	metrics, selection, backtest_selection	Scoring and selection rules
ml_service/app/monitoring	drift, performance, retraining	Evidently drift, MAE check, decision
ml_service/app/inference, caching, routes, schemas	production_model, prediction_cache, predict, schemas	Serving layer
ml_service/app/config.py, main.py	Central configuration; FastAPI entry point	All thresholds in one file
backend/	Django project: forecasts, model_registry, monitoring, users apps	REST API, Celery tasks, SQLite
src/, public/	React application: 8 pages, 3 API client modules	815 lines of JavaScript/JSX
tests/, backend/*/tests.py	25 ML test modules and 5 Django test modules	187 test functions in total

Path	Content	Notes
notebooks/	7 inspection scripts	Dataset inspection and variant comparison
data/, models/, docs/	Raw and processed CSV; model artifacts and registry.json; MODELING_FINDINGS.md	Raw data is never edited

The ML application package contains 2,056 lines of Python, the test directory 1,681 lines and the front end 815 lines of JavaScript, so the system is small enough to be understood in full while still covering the whole life cycle of a model.

3.5 Design principles

- **Single source of truth for decisions.** Every threshold, window, lag, parameter and path is defined in `config.py`. The file states that every decision made during data inspection lives there.
- **Never invent data.** Bad values become missing values (NaN); the target is never interpolated.
- **Fail loudly on bad input, fail open on optional services.** The metric functions raise errors on NaN or empty input, whereas the Redis cache is allowed to fail without breaking predictions.
- **The test set is for reporting, not for choosing.** Selection uses validation data and backtests only.
- **Models are immutable.** Each save creates a new timestamped folder.

4. Data Description and Exploratory Analysis

4.1 Candidate datasets and the choice of the primary dataset

Two datasets were available in the `data/raw` folder: `PowerLoad_Dataset.csv` (10,000 rows) and `weather and load dataset.csv` (8,779 hourly rows for Ahmedabad). The inspection scripts in the `notebooks` folder (`inspect_powerload.py`, `inspect_signal.py`, `inspect_gaps.py` and the Ahmedabad scripts) were used to decide which one is a credible forecasting target. The decision, recorded in the header comment of `config.py`, was to reject the `PowerLoad` file as the primary dataset.

Figure 7 reproduces the evidence for that decision. The hourly mean load of the `PowerLoad` file varies only between 496.2 and 503.5 kW across the 24 hours of the day, so there is no daily cycle. Its weather columns are spread almost uniformly over round ranges, which is typical of synthetic generation. The absolute correlation between its load and temperature, humidity and wind speed is at most 0.011, and its autocorrelation at lags of 1, 2, 3, 24, 48 and 168 hours lies between -0.017 and 0.006 . A forecasting model has nothing to learn from a series that behaves like random noise, and any score obtained on it would be meaningless.

4.2 The Ahmedabad dataset

The selected dataset contains hourly observations from 1 November 2021 05:00 to 1 November 2022 23:00. Each row has the date parts (year, month, day, hour), five weather variables and the target, the electric load in megawatts. After constructing a timestamp index, the timeline contains 8,779 consecutive hours and no missing hour.

Table 3: Variables in the Ahmedabad dataset (summary statistics are from the raw file, before cleaning)

Variable	Unit / meaning	Mean	Min	Max
irradiance	Solar irradiance (−999 = sensor code)	220.5	−999	1038.1
temperature	Air temperature, °C	27.37	6.95	47.65
dewpoint	Dew point, °C	14.26	−6.81	27.76
specific humidity	Specific humidity	11.72	2.20	23.80
wind speed	Wind speed	3.54	0.13	9.97
Electric Load (MW)	Target: hourly electric load	72.93	−0.05	187.33

The raw means for irradiance and load are affected by the invalid values described next; the cleaned data are used for all modelling.

4.3 Data quality and cleaning rules

Two kinds of invalid values were found and are handled by the `clean()` function of `data_loader.py`. Both rules convert the bad value to a missing value (NaN). The raw CSV is never modified, and the target is never interpolated, because an invented load value would make the evaluation dishonest.

Table 4: Data quality issues and the rule applied

Issue	Extent	Rule (config.py)
Irradiance equal to −999 (sensor "no measurement" code)	24 consecutive hours, 7 and 8 January 2022	IRRADIANCE_MISSING_CODE = −999 → NaN
Load ≤ 0 MW (meter dropout)	79 rows in 22 runs; the longest run is 8 hours	MIN_VALID_LOAD_MW = 0.0 → NaN
Missing or duplicate timestamps	0 missing hours; duplicates raise an error	<code>check_timeline()</code>

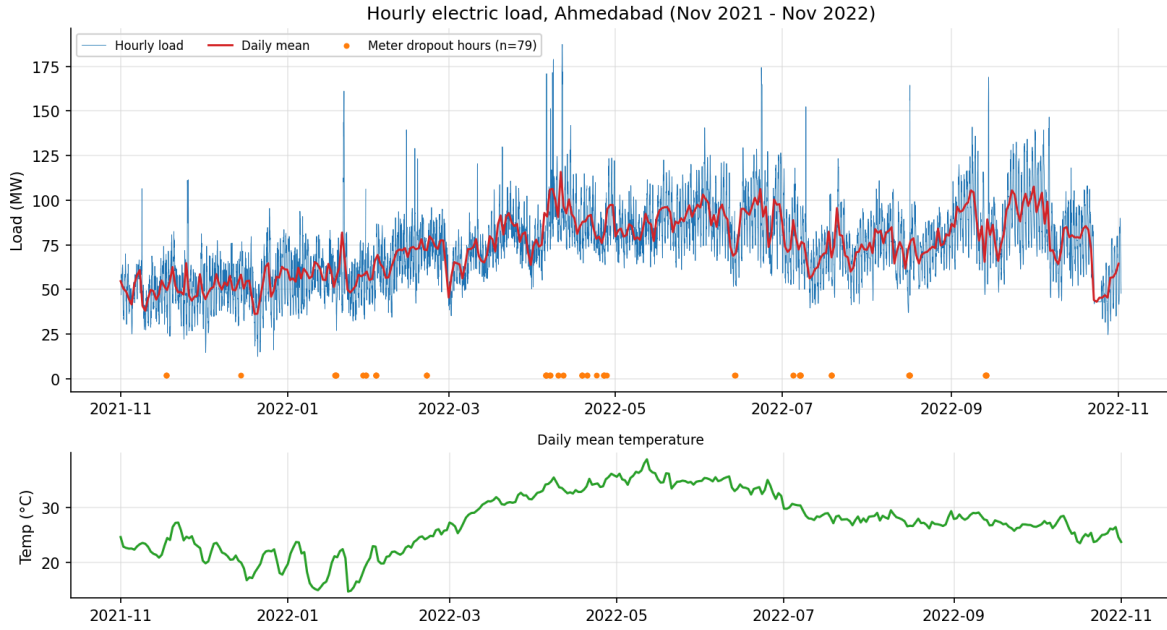


Figure 2: Hourly electric load, daily mean and daily mean temperature of Ahmedabad (November 2021 to November 2022); orange markers show the meter dropout hours.

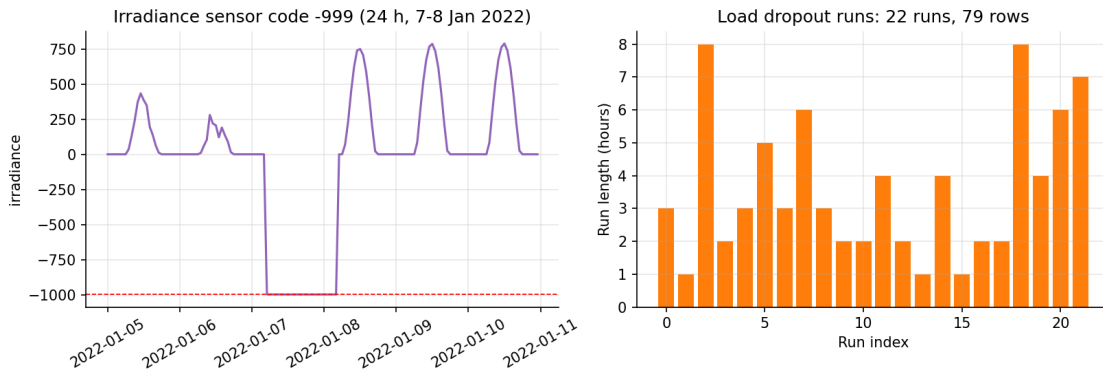


Figure 3: Left: the irradiance sensor code –999 over 24 hours. Right: lengths of the 22 load dropout runs (79 hours in total).

4.4 Daily, weekly and seasonal patterns

Figure 4 shows the three cycles. The mean load is lowest at 05:00 (57.6 MW) and highest at 15:00 (88.6 MW). Thursday, Friday and Saturday have the highest mean load (about 76 MW) and Sunday the lowest (69.5 MW). The monthly means are lowest from November to January and high in April, May, June and September (about 90 MW). The seasonal pattern follows the temperature curve in Figure 2. Because the data cover only one year, the seasonal effect cannot be separated from a possible year-to-year trend, which is a limitation that returns in Chapters 7 and 11.

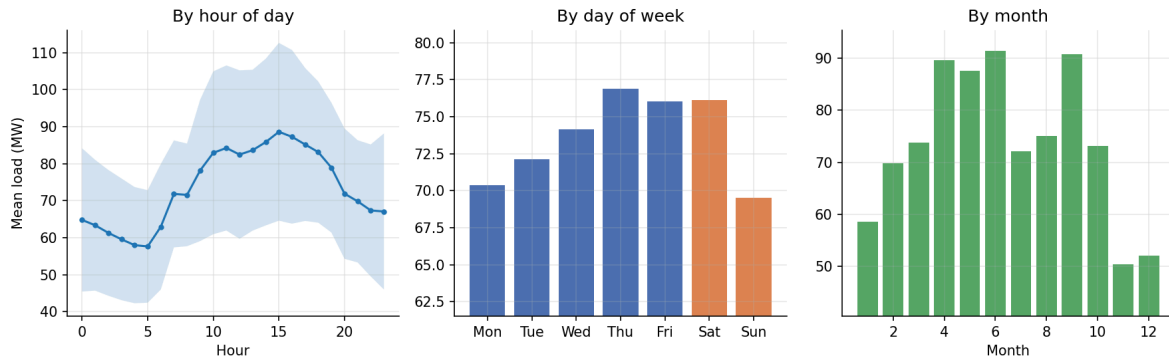


Figure 4: Mean load by hour of day (band = ± 1 standard deviation), by day of week and by month.

4.5 Relationship with the weather

The correlation matrix in Figure 5 shows that temperature has the strongest linear relationship with the load ($r = 0.66$), followed by irradiance (0.37), wind speed (0.29), specific humidity (0.23) and dew point (0.21). Temperature and dew point are themselves strongly related, so the models are expected to use the weather variables together. These weather values are the actual values at the target hour; in production they would be replaced by a weather forecast (Section 5.6).

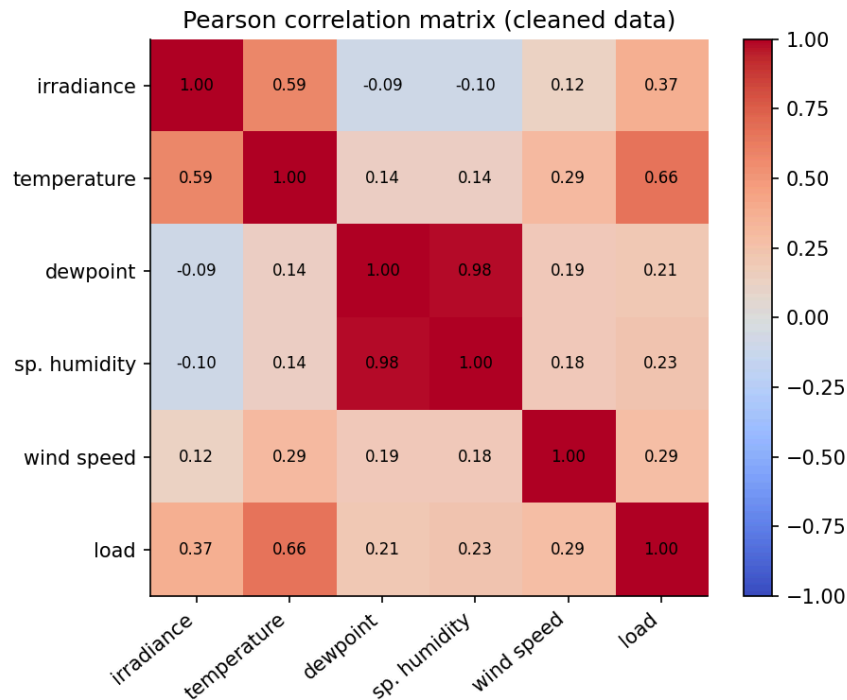


Figure 5: Pearson correlation matrix of the weather variables and the load (cleaned data).

4.6 Autocorrelation

The autocorrelation of the load is high: 0.94 at one hour, 0.82 at 24 hours, 0.76 at 48 hours and 0.67 at 168 hours (Figure 6). The peak near every multiple of 24 hours confirms the daily cycle. For a 24-hour-ahead forecast the one-hour autocorrelation cannot be used, because the load one hour earlier is unknown at forecast time. The 24-hour, 48-hour and 168-hour lags

are the strongest information that is legitimately available, and they determine the lag features of Chapter 5.

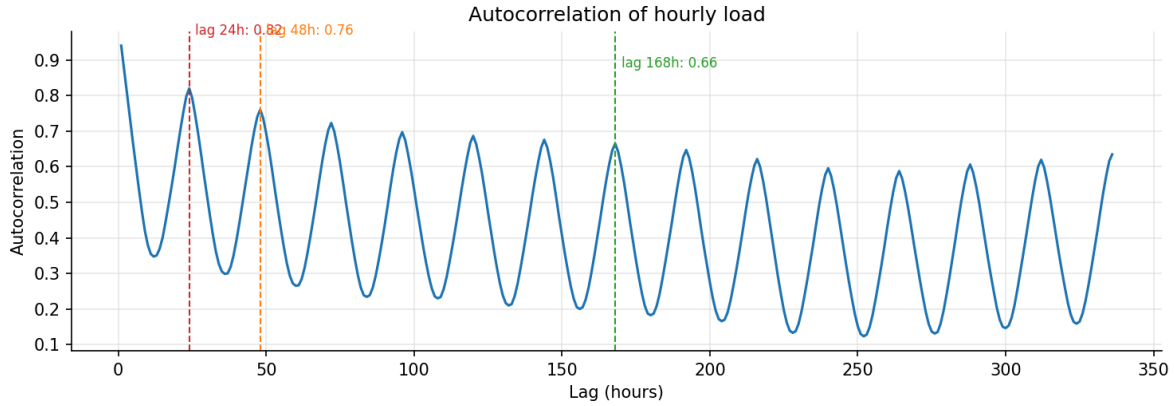


Figure 6: Autocorrelation function of the hourly load for lags up to 336 hours; the three lags used as features are marked.

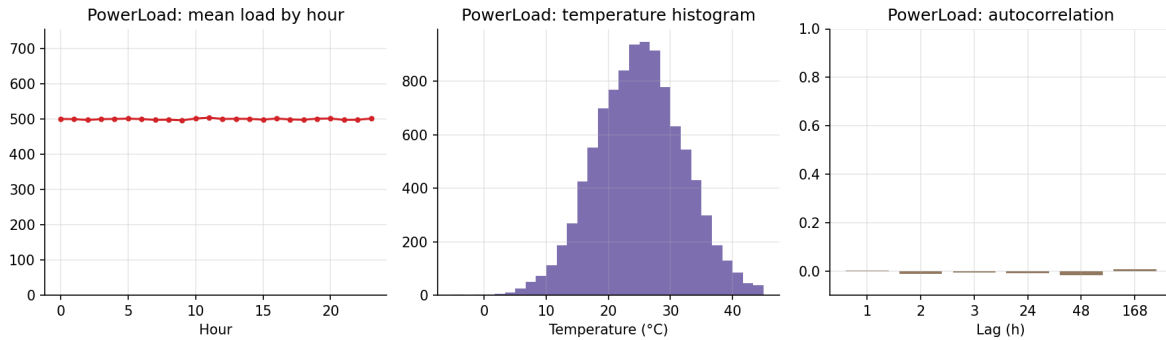


Figure 7: Evidence for rejecting PowerLoad_Dataset.csv: flat hourly profile, uniform-like temperature histogram and near-zero autocorrelation.

4.7 Level differences between the periods

Figure 8 compares the load level of the training, validation and test periods. The mean load is 72.19 MW in the training period, 73.63 MW in the validation period and 81.52 MW in the test period. The test period (September and October 2022) therefore has a noticeably higher average load than any earlier split. This level shift is central to the results in Chapter 7: the models are trained on a period whose average is about 9 MW lower than the period in which they are finally tested.

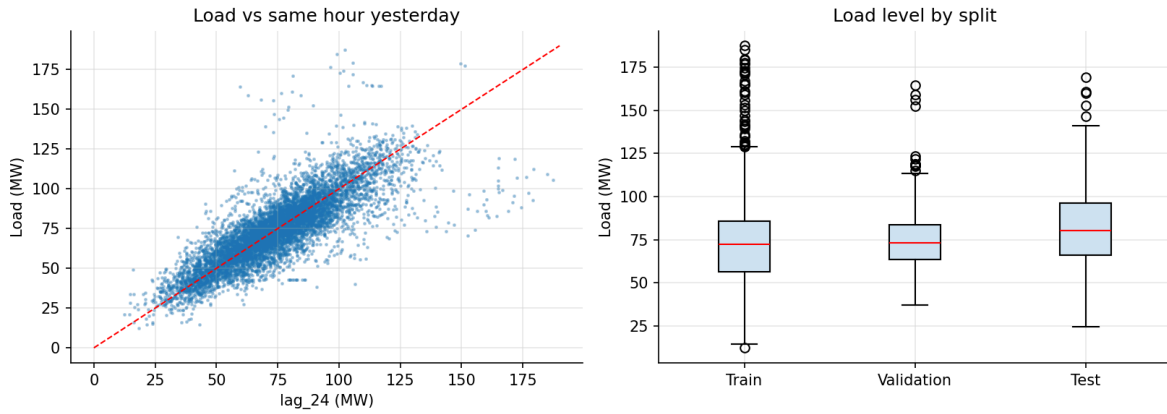


Figure 8: Left: load against the load of the same hour yesterday (dashed line = perfect repetition). Right: distribution of the load in each chronological split.

5. Data Preprocessing and Feature Engineering

This chapter explains the code in `ml_service/app/preprocessing`. The overall flow is shown in Figure 9: raw CSV, cleaning, calendar features, lag features, rolling features, removal of unusable rows and then the split and training steps described in Chapter 6.

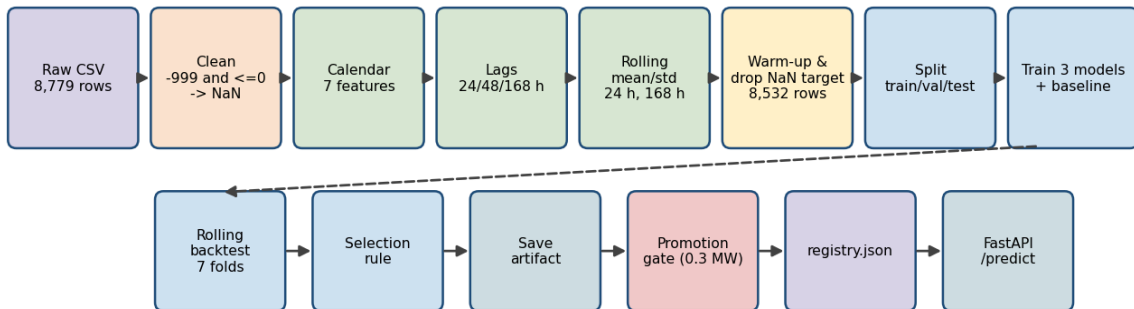


Figure 9: End-to-end ML pipeline of GridPredict, from the raw CSV to the production API.

5.1 Configuration (config.py)

Every decision is centralised in `config.py` (listed in Appendix B). The forecast horizon is `FORECAST_HORIZON_HOURS = 24`, the lags are `LAG_HOURS = [24, 48, 168]`, and the rolling windows are `ROLLING_WINDOWS_HOURS = [24, 168]`. Directly after the lag list the file contains an assertion that every lag is at least as long as the horizon, so that a configuration error that would leak the future stops the program at import time:

```
assert all(lag >= FORECAST_HORIZON_HOURS for lag in LAG_HOURS), (
    "Every lag must be >= FORECAST_HORIZON_HOURS, otherwise the future leaks into the features."
)
```

5.2 Loading, timeline check and cleaning (data_loader.py)

The loader performs four steps, each in its own function. `load_raw()` reads the CSV and raises an error if the file or a required column is missing. `build_timestamp_index()` combines YEAR, Month, Day and Hour into a single `DatetimeIndex` called `Timestamp` and keeps only the weather columns and the target. `check_timeline()` refuses to continue if there are duplicate timestamps, sorts the data if necessary and re-indexes it to a strictly hourly range, adding any missing hours as NaN rows. Finally `clean()` applies the two rules of Table 4:

```
bad_irradiance = df["irradiance"] == config.IRRADIANCE_MISSING_CODE
df.loc[bad_irradiance, "irradiance"] = float("nan")

bad_load = df[config.TARGET_COLUMN] <= config.MIN_VALID_LOAD_MW
df.loc[bad_load, config.TARGET_COLUMN] = float("nan")
```

The function logs how many values were changed (24 irradiance values and 79 load values on this dataset). No rows are deleted at this stage, which keeps the hourly timeline intact so that lags and rolling windows remain exact.

5.3 Calendar features (calendar_features.py)

Seven calendar features describe the target hour: `hour`, `day_of_week` (0 = Monday), `is_weekend`, `month`, `quarter`, `week_of_year` and `is_holiday`. All of them are known in advance, so they cannot leak information. The holiday flag is built with the `holidays` package for country IN and subdivision GJ (Gujarat), the state of Ahmedabad. The function raises a `TypeError` if the input does not have a `DatetimeIndex`.

5.4 Lag features (lag_features.py)

For a row at target hour t , `lag_h` is the load at exactly $t - h$ hours. The implementation looks values up by timestamp and not by row position, so the lag stays correct even if a row is missing:

```
shifted = series.shift(freq=pd.Timedelta(hours=lag))
out[lag_column_name(lag)] = shifted.reindex(out.index)
```

`shift(freq=...)` moves each value forward in time by `lag` hours, and `reindex` aligns the result to the rows of the table. Lags shorter than the horizon raise a `ValueError`, a missing load stays missing in the lag, and the input `DataFrame` is not modified. The project tests all of these behaviours (Chapter 10).

5.5 Rolling features (rolling_features.py)

The rolling statistics summarise the recent level and variability of the load. Their windows must end at $t - 24$, never at t . The module computes an ordinary trailing rolling statistic that

ends at each hour s and then shifts the result forward in time by 24 hours so that it lands on $t = s + 24$:

$$\begin{aligned} \text{rolling_mean_24 at } t &= \text{mean of the load from } t-47 \text{ h to } t-24 \text{ h} & \text{rolling_mean_168 at } t \\ &= \text{mean of the load from } t-191 \text{ h to } t-24 \text{ h} \end{aligned}$$

```
min_periods = max(2, math.ceil(w * config.ROLLING_MIN_FRACTION))
roll = series.rolling(f"{w}h", min_periods=min_periods, closed="right")
out[mean_col] = roll.mean().shift(freq=horizon).reindex(out.index)
out[std_col] = roll.std().shift(freq=horizon).reindex(out.index)
```

Missing loads are ignored inside a window, and a window needs at least 50 % of its hours to contain data (`ROLLING_MIN_FRACTION = 0.5`), so `min_periods` is 12 for the 24-hour window and 84 for the 168-hour window. The result is four features: `rolling_mean_24`, `rolling_std_24`, `rolling_mean_168` and `rolling_std_168`.

Leakage guard: every lag and rolling window ends at $t - 24$ h (hours relative to target t)

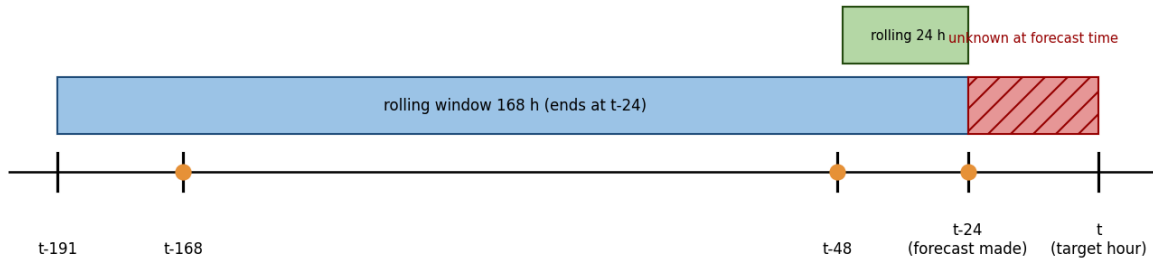


Figure 10: Leakage guard: the forecast is made at $t - 24$ h, so every lag (orange dots) and rolling window ends at or before that moment.

5.6 Assembling the feature table (pipeline.py)

`build_features()` runs the steps in a fixed order: calendar features, lags, rolling features, selection of the model columns, removal of the warm-up period and removal of rows whose target is missing. Row filtering is deliberately done last, so that lags and rolling windows are computed on the complete hourly timeline:

```
df = add_calendar_features(clean)
df = add_lag_features(df)
df = add_rolling_features(df)
df = df[feature_columns() + [config.TARGET_COLUMN]]
warmup_end = df.index.min() + pd.Timedelta(hours=max(config.LAG_HOURS))
df = df[df.index >= warmup_end]
df = df[df[config.TARGET_COLUMN].notna()]
```

The first 168 hours are removed because the longest lag needs a week of history. The table shrinks from 8,779 rows to 8,611 after the warm-up and to **8,532 rows** after dropping the 79 rows with a missing target; it is saved as `data/processed/ahmedabad_processed.csv` (first row 8 November 2021 05:00, last row 1 November 2022 23:00). Lag columns can still contain NaN where the earlier load was a dropout (79, 78 and 77 rows for `lag_24`, `lag_48` and `lag_168`) and irradiance has 24 NaN

values. The tree-based models used here accept missing values, and the baseline simply skips rows where its lag is unknown.

A note on the weather features: they are the **actual** weather at the target hour (WEATHER_ASSUMPTION = "actual_at_target_hour"). In a real deployment the same columns would be filled with a weather forecast, which has its own error, so the live accuracy would likely be lower than the figures in this report. This assumption is recorded in the metadata of every saved model.

Table 5: The 19 model input features (the Random Forest candidate uses the 16 features that do not carry a † mark)

Group	Features	Description
Calendar (7)	hour, day_of_week, is_weekend, is_holiday, month†, quarter†, week_of_year†	Known in advance; † = season-identifying columns removed in the "no_season" variant
Weather (5)	irradiance, temperature, dewpoint, specific humidity, wind speed	Actual value at the target hour (stands in for a forecast)
Lag (3)	lag_24, lag_48, lag_168	Load 1 day, 2 days and 1 week before the target hour
Rolling (4)	rolling_mean_24, rolling_std_24, rolling_mean_168, rolling_std_168	Mean and standard deviation of the load over windows ending at $t - 24$ h

6. Modelling Methodology

6.1 Chronological split (split.py)

The data are cut by date and never shuffled, so a model is always tested on a period that comes after its training data (Figure 11). Boundaries are inclusive and are defined in `config.py` as `TRAIN_END = "2022-06-30 23:00:00"` and `VALIDATION_END = "2022-08-31 23:00:00"`; the test set is everything after. The function raises an error if the index is unsorted, if the boundaries are in the wrong order or if any part is empty.

Table 6: Chronological split of the 8,532 modelling rows

Part	Period	Rows	Mean load (MW)
Train	8 Nov 2021 05:00 to 30 Jun 2022 23:00	5,583	72.19
Validation	1 Jul 2022 to 31 Aug 2022	1,468	73.63
Test	1 Sep 2022 to 1 Nov 2022 23:00	1,481	81.52

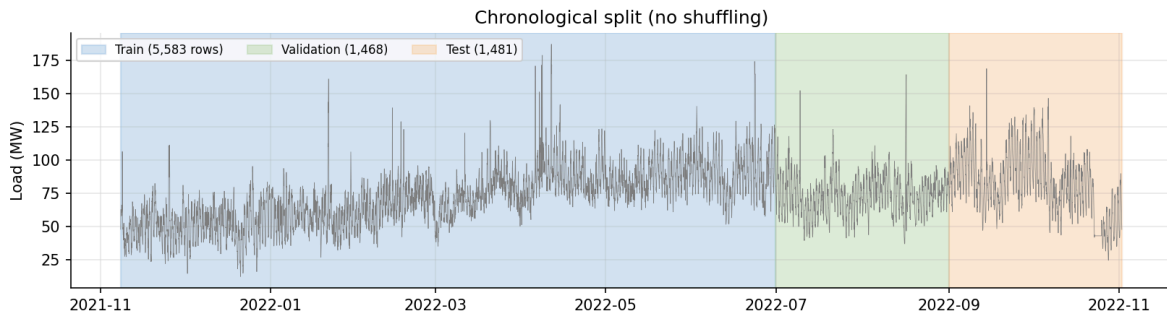


Figure 11: Chronological train, validation and test split of the hourly load.

6.2 Baselines (baseline.py)

Two naive baselines are defined. The `lag_24` baseline predicts the load as the load 24 hours earlier, and the `lag_168` baseline uses the load one week earlier. No learning is involved. Rows where the lag is unknown cannot be predicted by the baseline and are skipped and counted (20 skipped rows on validation and 7 on the test set). To keep the comparison fair, every model and the baseline are always scored on the same rows, namely those where the baseline can predict.

6.3 Models and settings (model_factory.py)

`model_factory.py` builds a new, untrained model by name (`random_forest`, `xgboost` or `lightgbm`) using the parameters in `config.py`, all with `random_state = 42` for reproducibility. The settings are sensible defaults and are not tuned; the configuration file states that any later tuning must use the validation set only.

Table 7: Model settings from config.py

Parameter	Random Forest	XGBoost	LightGBM
n_estimators	300	300	300
learning_rate	n/a	0.05	0.05
Tree size control	min_samples_leaf = 2	max_depth = 6	num_leaves = 31, min_child_samples = 20
Feature sampling	max_features = 0.5	colsample_bytree = 0.8	colsample_bytree = 0.8
Row sampling	bootstrap (default)	subsample = 0.8	subsample = 0.8, subsample_freq = 1
Other	n_jobs = -1	tree_method = hist, n_jobs = -1	deterministic = True, n_jobs = -1, verbose = -1

6.4 Training and scoring (trainer.py and metrics.py)

`train_model()` checks that the training data are non-empty, that X and y have the same length and that y has no missing value, then builds and fits the model. `predict()` returns a Series that keeps the Timestamp index, and `score_model()` can restrict scoring to a subset of rows. The four metrics (MAE, RMSE, MAPE and R^2) are implemented in `metrics.py`. Their shared helper `_prepare()` converts the inputs to float arrays and raises a `ValueError` for different lengths, empty input or NaN and infinite values, so a misleading score is never produced silently. MAPE raises an error when an actual value is zero, and R^2 when all actual values are identical.

6.5 Comparison and the first selection rule (compare.py and selection.py)

`compare_models()` scores the baseline and all three models on the validation set and calls `select_best_model()`. The rule, documented in `config.py`, is:

1. a model is eligible only if its validation MAE is strictly below the baseline MAE;
2. among eligible models the lowest MAE wins;
3. eligible models within `SELECTION_TIE_TOLERANCE_MW = 0.1` MW of the best MAE are tied, and the lowest RMSE among them wins;
4. if no model is eligible there is no winner and nothing is saved.

The returned `SelectionResult` also holds a human-readable reason and a table sorted by MAE. For the actual run the reason was: "random_forest, xgboost are within 0.1 MW of the best MAE (8.47 MW); xgboost has the lowest RMSE (11.46 MW)."

6.6 Rolling-origin backtest (backtest.py)

A single validation window of two months is a weak basis for selection, as the results will show. `backtest()` implements an expanding-window rolling-origin backtest. For each calendar month M from February 2022 to August 2022 (seven folds) it trains on all rows before the start of M minus a 24-hour gap and tests on month M . The gap equals the forecast

horizon, because labels from the last 24 hours before M would not yet be known when the forecast for the first hour of M is made. The function uses only data up to `VALIDATION_END`, so the test period is never touched, and each fold scores the model and the `lag_24` baseline on the same rows. It also records the model bias (mean of prediction minus actual) for each fold.

Two variants can be selected through arguments: a `features` list (used to drop the season columns `month`, `quarter` and `week_of_year`) and `target_mode`, where "delta_baseline" trains the model to predict the change from yesterday's load and adds the lag back afterwards. The script `notebooks/compare_variants.py` compares four variants (full, no_season, full_delta and no_season_delta) for the three models.

6.7 The second selection rule (backtest_selection.py)

`select_from_backtests()` applies a rule that was fixed before the comparison was run: a candidate is eligible if it beats the baseline in at least 70 % of the folds (5 of 7) **and** its mean MAE is lower than the baseline mean MAE. The lowest mean MAE among eligible candidates wins; candidates within 0.1 MW of the best are tied, and the one that beats the baseline in the most folds wins. The function computes the number of folds needed with `math.ceil(round(n_folds * min_fold_fraction, 9))`, because in floating point 10×0.7 equals 7.000000000000001 and a plain `ceil` would demand 8 folds. Figure 12 summarises both rules and the promotion gate.

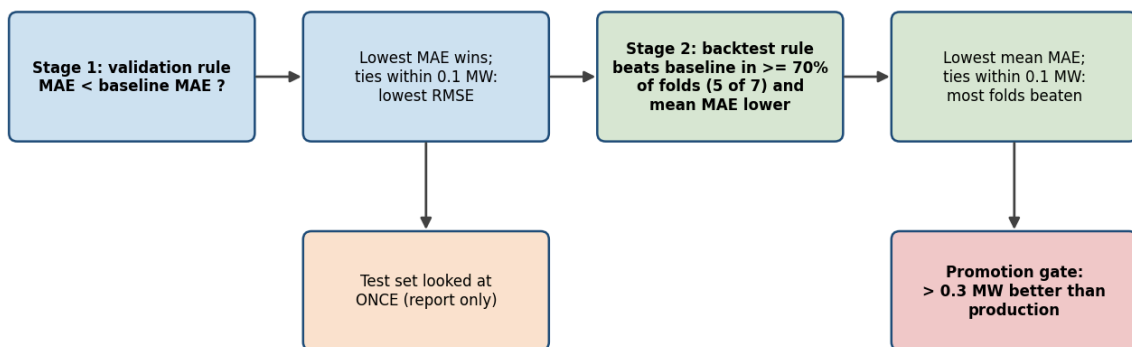


Figure 12: Selection logic: validation rule, backtest rule, single look at the test set and the promotion gate.

6.8 Training runs and artifacts (train_pipeline.py, train_candidate.py, artifacts.py)

`run_training()` executes the full flow: features, split, comparison, selection, one scoring of the winner on the test set and saving. The winner is trained on the training split only and is saved with status "candidate". If no model beats the baseline, nothing is saved. `train_candidate.py` trains the backtest-selected candidate (Random Forest without season columns) on train plus validation data, saves it and reports its test score once. Its

metadata states that the test score is a report only and that the test period had already been viewed once for an earlier candidate, so the number is not a clean unseen evaluation.

`save_artifact()` creates a new folder `models/<model_name>_<UTC timestamp>/` containing `model.joblib` and `metadata.json`, and refuses to reuse an existing folder (`exist_ok=False`). The metadata records the feature list, the training dates, the model parameters, the scores and the selection reason. `load_artifact()` reads both files back and raises an error if either is missing.

6.9 Registry and promotion gate (registry.py)

The file `models/registry.json` is tracked in Git, unlike the model files, and holds the current production entry and the full history of promotion decisions. Before anything is promoted the production model is the naive baseline `naive_lag_24`. `evaluate_promotion()` returns a decision record in which `improvement = production_mae - challenger_mae` and `promoted = improvement > 0.3 MW` (`PROMOTION_MIN_MAE_IMPROVEMENT_MW`). `promote()` applies the decision against the current registry, appends it to the history whether or not it won, replaces the production entry only if it won, and saves. It refuses to continue if the production entry has no recorded MAE, because a fair comparison would then be impossible.

7. Experiments and Results

This chapter reports the experiments in the order in which they were carried out. All numbers are taken from the project files: the validation and test scores come from the `metadata.json` files in the `models` folder and from `docs/MODELING_FINDINGS.md`, and the rolling-backtest folds, the analysis of the test predictions and the feature importances were recomputed from the saved processed data and the saved Random Forest artifact. Where a value was recomputed rather than copied, the text says so.

7.1 Baselines and the validation comparison

Before any learning, the naive baselines were scored on the validation period (July and August 2022). "Same hour yesterday" (`lag_24`) reached an MAE of 9.25 MW and an R^2 of 0.220, while "same hour last week" (`lag_168`) reached an MAE of 12.36 MW and an R^2 of -0.187 , which is worse than always predicting the mean. The `lag_24` baseline therefore became the bar that every model has to beat. Table 8 and Figure 13 show the validation results for the three untuned models.

Table 8: Validation scores (July to August 2022; baseline scored on 1,448 rows, 20 rows skipped)

Model	MAE (MW)	RMSE (MW)	MAPE (%)	R^2
Baseline <code>lag_24</code>	9.25	13.29	12.78	0.220
Random Forest	8.52	11.58	12.64	0.407
XGBoost	8.47	11.46	12.39	0.420

Model	MAE (MW)	RMSE (MW)	MAPE (%)	R ²
LightGBM	8.61	11.67	12.68	0.399

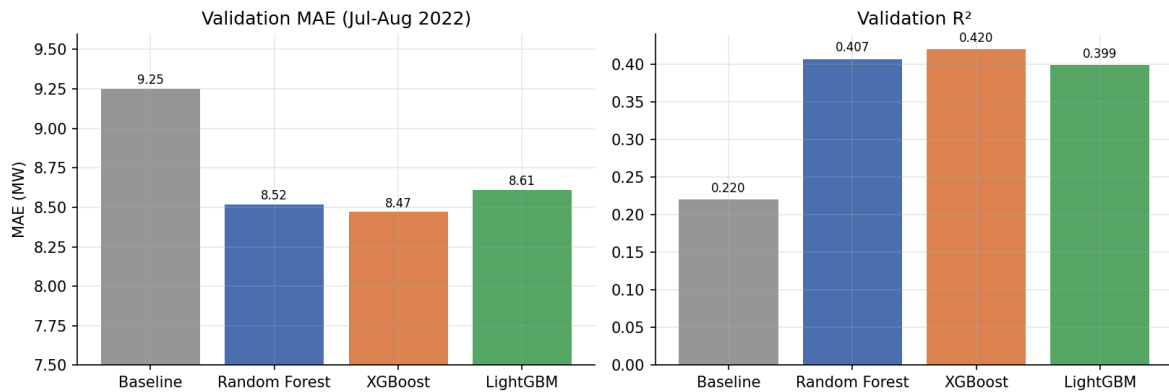


Figure 13: Validation MAE and R² of the baseline and the three models.

All three models beat the baseline on validation, by between 0.64 and 0.78 MW (about 7 % to 8 %). Random Forest and XGBoost were within 0.1 MW of the best MAE, so under the first selection rule the tie was broken by RMSE and XGBoost was selected. At this point the result looked successful, which is exactly why the next step is important.

7.2 The single-window selection fails on the test set

XGBoost was trained on the training split and scored once on the held-out test period (September to November 2022). It lost to the baseline on every metric (Table 9, Figure 14).

Table 9: Test-set scores of the baseline and the XGBoost model selected on validation (baseline: 1,474 rows, 7 skipped)

Scored on	MAE (MW)	RMSE (MW)	MAPE (%)	R ²
Baseline lag_24	9.01	12.54	11.83	0.713
XGBoost, same rows as baseline	10.67	13.42	13.79	0.671
XGBoost, all test rows	10.81	13.89	13.85	0.657

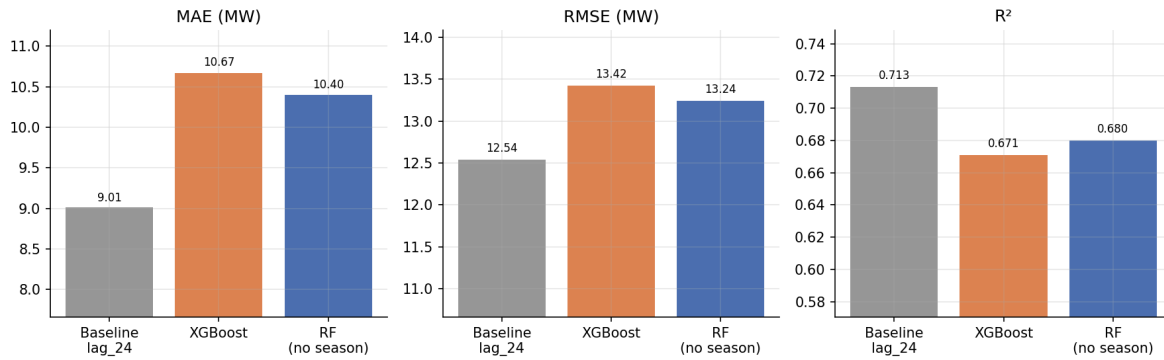


Figure 14: Test-set MAE, RMSE and R^2 of the baseline, XGBoost (selected on validation) and the Random Forest candidate (selected by backtest).

XGBoost was worse than the baseline by 1.66 MW (18 %) although it had been better by 0.78 MW on validation. Two lessons follow. First, a single two-month validation window is not a reliable basis for choosing between models. Second, the R^2 values are not comparable between periods: they are low on validation (0.22 to 0.42) and high on test (0.66 to 0.71) because R^2 is relative to the variance of the period being scored, and the test period has a larger spread of load values. The MAE, which is in MW, is the more honest comparison across periods.

7.3 Rolling backtest and re-selection

To obtain a more robust comparison, the seven-fold rolling-origin backtest of Section 6.6 was built, covering February to August 2022 and never touching the test set. The results documented in `MODELING_FINDINGS.md` are summarised in Table 10 and Figure 15.

Table 10: Rolling backtest summary (7 monthly folds, documented project results)

Model	Folds beating baseline	Mean MAE (MW)	Eligible (≥ 5 of 7 and mean MAE < baseline)
Random Forest	6 / 7	8.54	Yes
XGBoost	4 / 7	8.72	No
LightGBM	2 / 7	9.15	No
Random Forest, no season columns	6 / 7	8.44	Yes (best combination)

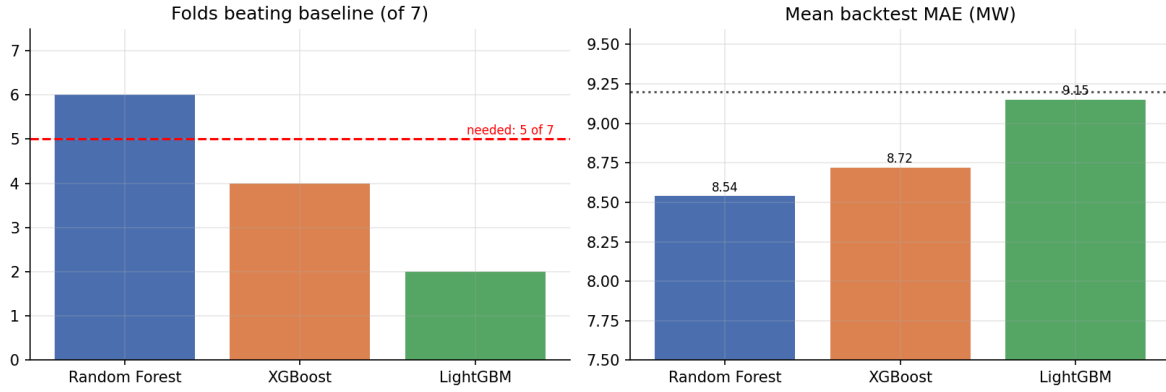


Figure 15: Rolling backtest: number of folds in which each model beats the baseline (left) and mean MAE (right).

XGBoost, which had won on validation, beat the baseline in only 4 of 7 folds, and LightGBM in only 2 of 7. The Random Forest was the only model that was consistently better. Two variants were also tested under a rule fixed before running the comparison: removing the calendar columns that only identify the season (month, quarter, week_of_year), and predicting the change from yesterday's load instead of the raw value. Random Forest without the season columns was the best eligible combination.

7.3.1 Independent recomputation of the Random Forest folds

To verify the documented figures and to show the behaviour fold by fold, the backtest logic of `backtest.py` was re-run with the same Random Forest settings on the saved processed data (using scikit-learn 1.8.0, whereas the project environment pins 1.9.1). The recomputed mean MAE was 8.55 MW for all features and 8.42 MW without the season columns, against 9.02 MW for the baseline, with 6 of 7 folds won in both cases. The documented values are 8.54 MW and 8.44 MW, so the small differences (at most 0.02 MW) are consistent with a library-version difference. XGBoost and LightGBM are not recomputed here, and their numbers in Table 10 are the project's documented values.

Table 11: Recomputed per-fold backtest (Random Forest; MAE and bias in MW; bias = mean of prediction minus actual)

Fold	Train rows	Test rows	Actual mean	Baseline MAE	RF all MAE	RF no-season MAE	RF no-season bias
2022-02	1994	656	69.63	8.01	10.17	9.50	-7.16
2022-03	2658	744	73.80	8.28	8.16	8.07	+0.42
2022-04	3402	670	89.20	9.81	9.62	9.61	-6.84
2022-05	4097	744	87.52	7.71	6.78	6.83	+2.38
2022-06	4841	716	91.45	10.83	8.65	8.65	-1.28
2022-07	5559	716	71.98	9.00	8.72	8.51	+4.34
2022-08	6289	732	74.97	9.50	7.74	7.76	-1.09

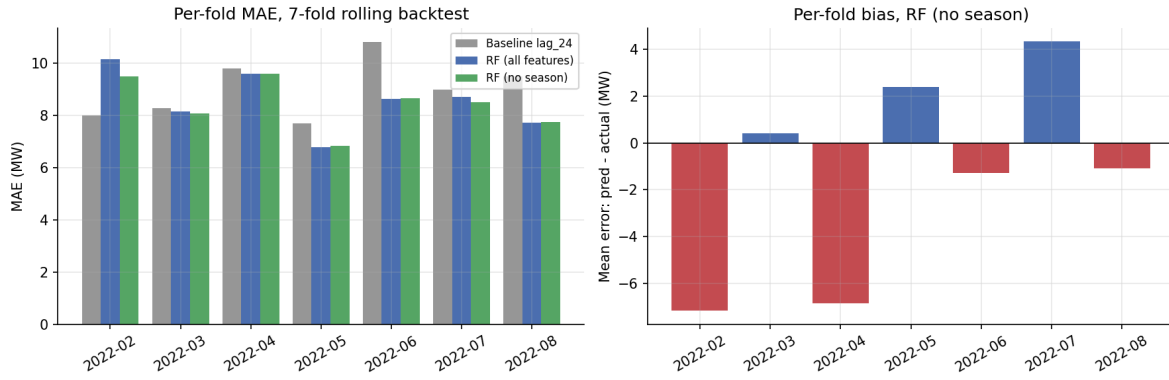


Figure 16: Left: per-fold MAE of the baseline and the two Random Forest variants. Right: per-fold bias of the Random Forest without season columns.

The only fold in which the baseline clearly won is February 2022 (8.01 MW against 9.50 MW). The bias column shows a pattern: the model underpredicts (negative bias) in February and April, the months in which the average load rose sharply from the previous month (from 58.6 to 69.8 MW in the monthly means of Figure 4, and from 73.8 MW in March to 89.2 MW in April in Table 11), and it overpredicts (positive bias) in July, after the high-load month of June (91.5 MW in Table 11) was followed by a fall to 72.0 MW. In other words, the model tends to lag behind changes in the load level. This is an early warning of the behaviour that causes the failure on the test set.

7.4 The backtest-selected candidate on the test set

The candidate (Random Forest, 16 features without the season columns) was trained on all data up to the end of validation (7,051 rows, 8 November 2021 to 31 August 2022) and scored on the test set. The score is a labelled report only; the metadata of the model states that the test period had already been viewed once, so this is not a clean unseen evaluation, but the score was not used to choose or tune anything.

Table 12: Test-set scores of the Random Forest candidate (no season columns)

Scored on	MAE (MW)	RMSE (MW)	MAPE (%)	R ²
Baseline lag_24 (1,474 rows)	9.01	12.54	11.83	0.713
Random Forest, same rows	10.40	13.24	13.25	0.680
Random Forest, all 1,481 rows	10.55	13.73	13.32	0.665

Random Forest also lost to the baseline, by 1.39 MW (15 %), a smaller margin than XGBoost (1.66 MW). Loading the saved model artifact and predicting the test rows reproduces the reported MAE of 10.40 MW exactly, which confirms that the saved model and the processed data are consistent with the metadata.

7.4.1 Where do the errors come from?

Figure 17 compares the forecasts with the actual load. The Random Forest follows the daily shape well but tends to sit below the actual load in the high-load days of September and early

October, and to stay above it during the abrupt drop around 22 October. Figure 18 shows the error structure in more detail.

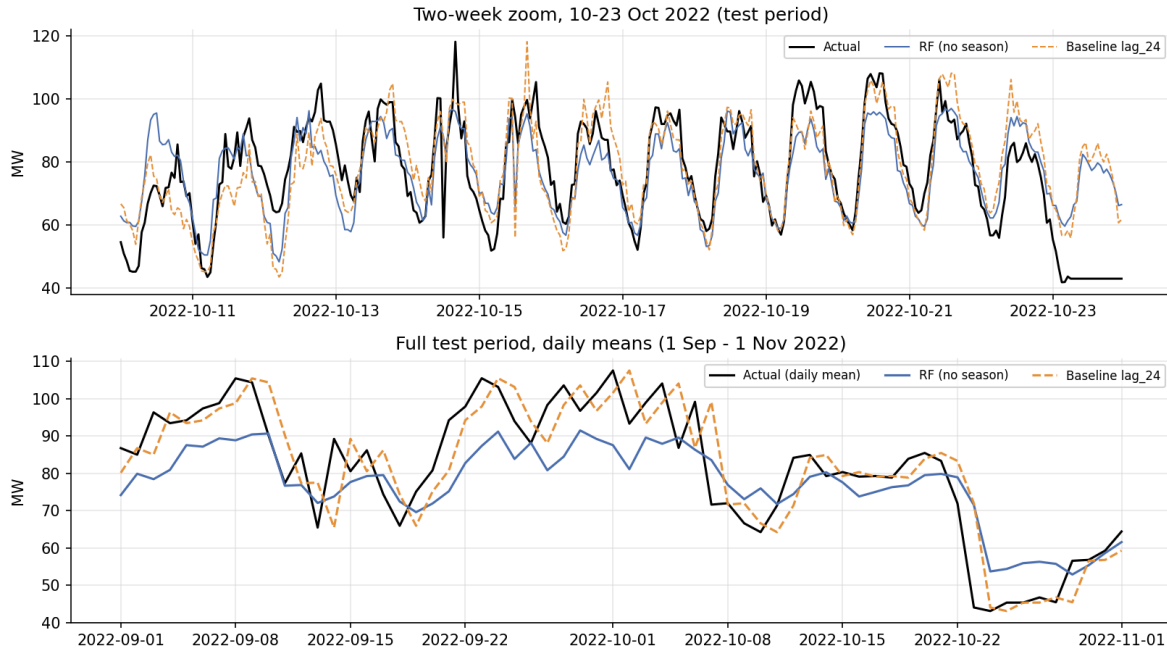


Figure 17: Test period: a two-week zoom (top) and the daily means over the whole test period (bottom) for the actual load, the Random Forest and the baseline.

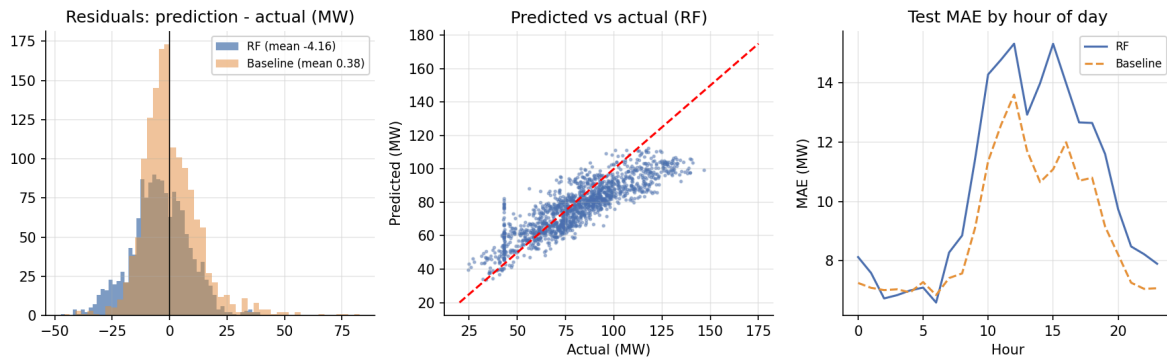


Figure 18: Left: distribution of forecast errors. Centre: predicted against actual load for the Random Forest. Right: test MAE by hour of day.

The recomputed error analysis gives the following facts. The mean error of the Random Forest is -4.16 MW (it underpredicts on average), whereas the baseline is almost unbiased ($+0.38$ MW). The scatter plot shows that the Random Forest predictions flatten near 100 to 110 MW; its highest prediction in the test period is 112.3 MW, while the highest actual load is 168.9 MW. The MAE by hour shows that the Random Forest error is largest from about 09:00 to 18:00, which is when the load is highest. Splitting the rows by actual load makes the effect explicit (Table 13).

Table 13: Test-set error by load level (recomputed from the saved model; rows where the baseline can predict)

Rows	Count	RF MAE (MW)	Baseline MAE (MW)	RF mean error (MW)
Actual load $\leq$ 100 MW	1,182	8.09	8.60	not computed
Actual load $>$ 100 MW	292	19.77	10.69	-19.56
All rows	1,474	10.40	9.01	-4.16

This is the key result of the project. On the 80 % of hours with load up to 100 MW the Random Forest is actually slightly better than the baseline (8.09 MW against 8.60 MW, about 6 %). On the 20 % of hours with load above 100 MW it is much worse (19.77 MW against 10.69 MW) and underpredicts by almost 20 MW on average. These hours dominate the overall MAE.

7.4.2 Interpretation

The evidence is consistent with the explanation given in `MODELING_FINDINGS.md`. The test period has a higher average load (81.5 MW) than any earlier period (72.2 and 73.6 MW), and the model is trained mostly on months with a lower load level. A tree ensemble averages the targets of historical rows that resemble the current row; it cannot produce values beyond what such rows contained and tends to move towards typical historical values, so it follows a level shift slowly (Hastie, Tibshirani and Friedman, 2009). The naive baseline copies a recent actual value and therefore tracks any level shift automatically. This is a hypothesis supported by the bias pattern in the backtest (Table 11), the plateau in Figure 18 and the split of errors in Table 13; it has not been proved by an experiment that removes the shift, and the dataset cannot separate a seasonal effect from a trend because it covers only one year.

7.5 Feature importance

The saved Random Forest exposes impurity-based feature importances, which the Django endpoint `/api/feature-importance/` serves to the dashboard. Figure 19 and Table 14 show them. The three lag features together account for 63 % of the total importance (lag_24 alone 43.7 %), temperature is the most important weather variable (12.3 %), and the calendar variables `hour`, `is_weekend` and `is_holiday` have very small importance because the lags and rolling statistics already carry the daily and weekly pattern.

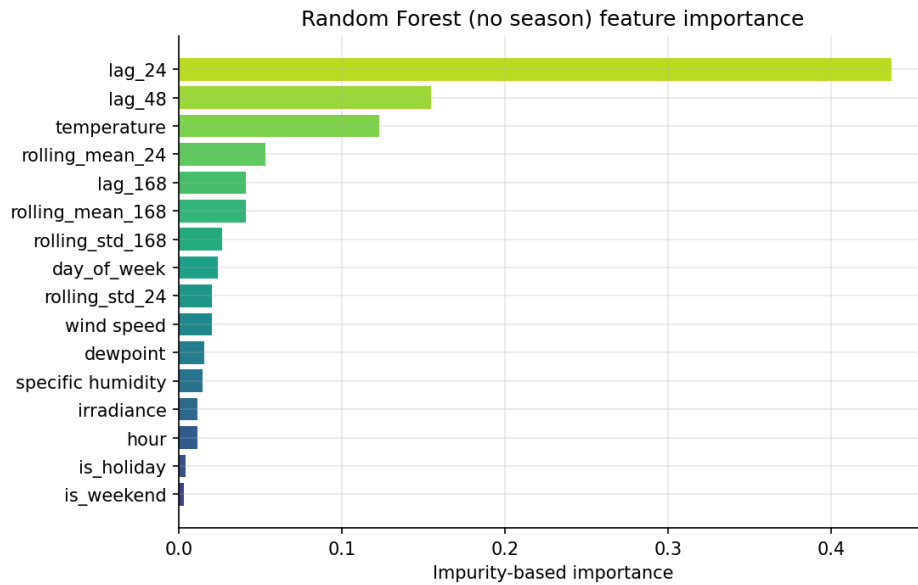


Figure 19: Impurity-based feature importance of the Random Forest candidate (no season columns).

Table 14: Top feature importances of the saved Random Forest model

Rank	Feature	Importance
1	lag_24	0.4369
2	lag_48	0.1545
3	temperature	0.1227
4	rolling_mean_24	0.0531
5	lag_168	0.0413
6	rolling_mean_168	0.0412
7	rolling_std_168	0.0267
8	day_of_week	0.0241

Impurity-based importances should be read with care: they describe how much a feature reduced the training error inside the trees, they are shared among correlated features, and they do not say how the model would behave under a level shift. They are shown for transparency and not as evidence of causation.

7.6 The promotion gate in action

Both candidates were passed through the promotion gate against the production baseline, using the baseline's test MAE of 9.01 MW as the production figure. The requirement was an improvement of more than 0.3 MW. Both challengers were rejected, and the registry recorded both decisions (Table 15, Figure 20).

Table 15: Promotion history in models/registry.json (decisions recorded on 25 September 2026)

Challenger	Challenger MAE	Production MAE	Improvement (MW)	Promoted
xgboost_full	10.67	9.01	-1.66	No

Challenger	Challenger MAE	Production MAE	Improvement (MW)	Promoted
random_forest_no_season	10.40	9.01	-1.39	No

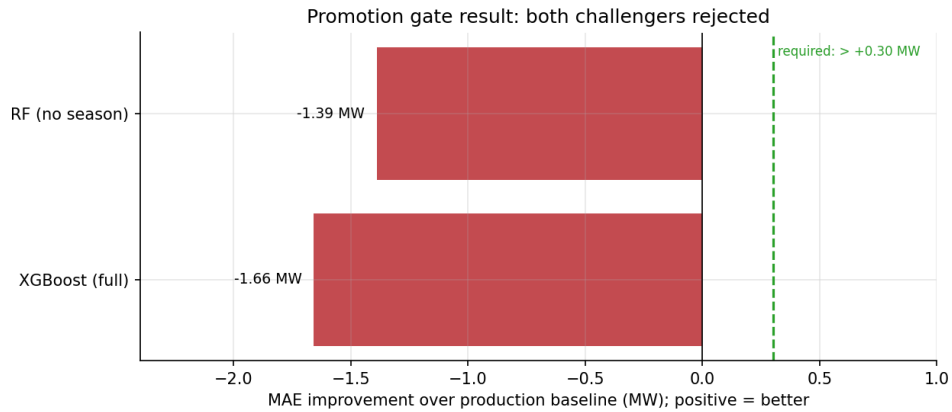


Figure 20: Promotion gate: MAE improvement of each challenger over the production baseline compared with the required margin of +0.30 MW.

The production entry in the registry is the baseline `naive_lag_24` with a recorded MAE of 9.01 MW. This is the intended behaviour of the design: the pipeline refused to deploy a model that would have made the forecast worse by 1.39 MW or more, even though the model had looked better than the baseline in validation and in the backtest.

7.7 Summary of the results

Table 16: Summary of all model evaluations (MAE in MW)

Stage	Baseline	Model	Outcome
Validation, Jul–Aug 2022	9.25	XGBoost 8.47	Model better; XGBoost selected
Test, Sep–Nov 2022	9.01	XGBoost 10.67	Model worse by 1.66 MW; not promoted
Backtest mean, Feb–Aug 2022	9.02 (recomputed)	RF no-season 8.44 (documented)	Model better in 6 of 7 folds; RF selected
Test, Sep–Nov 2022	9.01	RF no-season 10.40	Model worse by 1.39 MW; not promoted

8. Monitoring and Retraining

8.1 Design

A model that is in production must be watched. The monitoring package, `ml_service/app/monitoring`, combines two independent checks into one retraining decision (Figure 21). The requirement stated in the source code is that retraining must be based on statistically meaningful evidence, drift and/or performance degradation, and never on the fact that data changed. Each check has its own documented threshold in `config.py`.

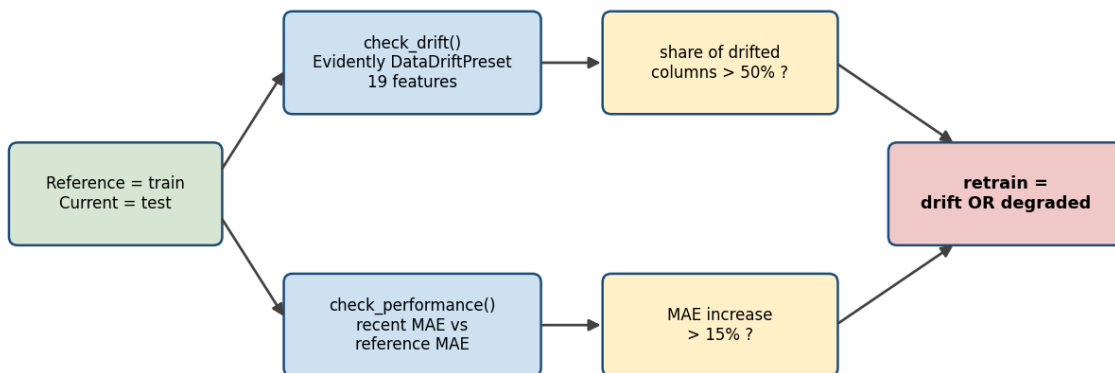


Figure 21: Retraining decision: the drift check and the performance check are combined with a logical OR.

8.2 Data drift (drift.py)

`check_drift(reference, current, columns, drift_share_threshold)` wraps the `Evidently Report([DataDriftPreset()])` so that the rest of the code works with a plain dictionary. Evidently selects a drift-detection method for each column according to its type. The module records an important subtlety: different method families point in opposite directions. Distance methods (normalised Wasserstein, Jensen-Shannon, population stability index) flag drift when the value is above the threshold, whereas p-value methods (Kolmogorov-Smirnov, Z-test, chi-square) flag drift when the value is below it. Because the preset's own pass/fail list was empty in the installed version, the code determines the flag itself with the table `_METHOD_DIRECTION`, and raises a `ValueError` for an unknown method instead of trusting an unverified result:

```

_METHOD_DIRECTION = {
    "Wasserstein distance (normed)": "above",
    "Jensen-Shannon distance": "above",
    "Population Stability Index": "above",
    "K-S p_value": "below",
    "Z-test p_value": "below",
    "Chi-square p_value": "below",
}

```

The share of drifted columns is then compared with `DRIFT_SHARE_THRESHOLD = 0.5`. The dataset is considered drifted only if more than half of the columns drifted, which avoids "retrain whenever data changes" because of a couple of unstable columns. The function returns the dataset flag, the share, the count, the number of columns, the threshold and a per-column dictionary of method, threshold, score and flag. It raises errors for missing columns and for empty inputs.

8.3 Performance degradation (performance.py)

`check_performance(y_true, y_pred, reference_mae, degradation_threshold)` compares the recent error with an explicit, documented reference MAE (in the project, the production model's known test MAE of 9.01 MW) rather than with the training error, which would always look better than later results. The relative increase is $(\text{recent MAE} - \text{reference MAE}) / \text{reference MAE}$, and performance is "degraded" when it exceeds `PERFORMANCE_DEGRADATION_THRESHOLD = 0.15`, i.e. 15 %. A relative threshold is used because the MAE naturally varies with the load level of the season.

8.4 The combined decision (retraining.py)

`decide_retraining()` runs both checks, sets `retrain = drifted or degraded` and builds a readable reason, for example "Retraining recommended: data drift detected (74 % of 19 features drifted, threshold 50 %)." If neither condition holds, the reason states the drift share and the relative performance change.

8.5 Results of the monitoring run

The scheduled task compares the training period (reference) with the test period (current) for the 19 model features. The result stored in the Django database (table `monitoring_data`), one record, checked on 26 September 2026 at 05:20 UTC) is: 14 of 19 features drifted (73.7 %), which is above the 50 % threshold; the recent MAE was 9.0119 MW against the reference 9.01 MW (a relative increase of 0.02 %, far below 15 %); performance is not degraded; and retraining is recommended because of drift alone.

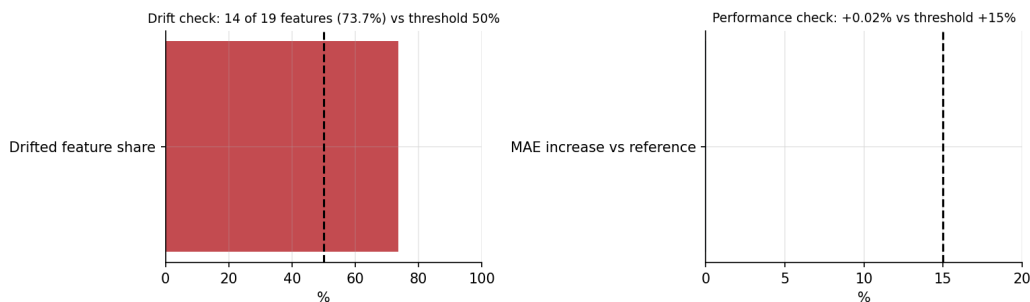


Figure 22: Stored monitoring result: the drift share exceeds its threshold, the performance change is negligible.

Evidently itself is not installed in the environment used to prepare this report, so the per-feature scores were recomputed with SciPy using the same definitions: the Wasserstein distance normalised by the standard deviation of the reference for numeric features, and the Jensen-Shannon distance for the binary or categorical columns `is_weekend`, `is_holiday` and `quarter`, each compared with 0.1. This recomputation flags exactly 14 of 19 features, the same count as the stored result, and is shown in Figure 23 and Table 17. It is an approximation of the Evidently output and not a copy of it, so the exact scores may differ slightly.

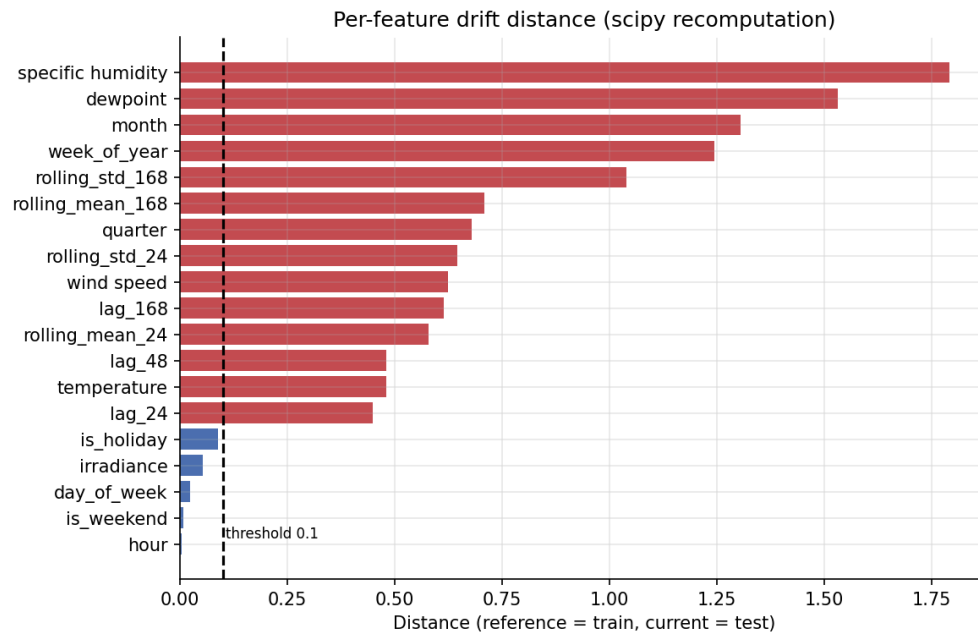


Figure 23: Per-feature drift distance between the training period (reference) and the test period (current), recomputed with SciPy; red bars exceed the 0.1 threshold.

Table 17: Drift flags by feature group (SciPy recomputation; 14 of 19 features exceed 0.1)

Group	Drifted	Not drifted
Calendar	month (1.31), week_of_year (1.25), quarter (0.68)	hour, day_of_week, is_weekend, is_holiday (0.09)
Weather	specific_humidity (1.79), dewpoint (1.53), wind speed (0.62), temperature (0.48)	irradiance (0.05)
Lag and rolling load	lag_24 (0.45), lag_48 (0.48), lag_168 (0.62), rolling_mean_24 (0.58), rolling_std_24 (0.65), rolling_mean_168 (0.71), rolling_std_168 (1.04)	none

8.5.1 Interpretation

The drift is real but largely expected. The reference period runs from November to June, while the current period is September to November, so the season-identifying calendar columns and the weather variables differ by construction, and the lag and rolling features have drifted because the load level is higher (Section 4.7). The hour of day, the weekday pattern and the holiday flag, which describe the daily routine rather than the season, are

stable. The alert therefore correctly shows that the input distribution has moved, but it does not show that the forecast is worse: the production model is the naive baseline, which uses only `lag_24`, and its error is unchanged. This exposes a design question that is discussed in Chapter 11: with an OR rule and a reference window that is not seasonally matched, a seasonal change alone can recommend retraining.

8.6 Scheduling with Celery (monitoring/tasks.py and migration 0002)

The task `run_drift_and_performance_check` is a Celery `shared_task`. It builds the features, splits them, takes the `lag_24` values of the test period as the production predictions, calls `decide_retraining()` with `reference_mae = 9.01`, and stores the result in the `DataMonitoring` model. The schedule is created by a Django migration (`0002_schedule_daily_drift_check.py`), which creates an interval schedule of one day and a periodic task named "Daily drift and performance check". Using a migration rather than manual clicks in the admin keeps the schedule reproducible and tracked in Git. The docstring of the task records an honest limitation: it currently runs against the static train/test split, because there is not yet a live stream of new data; once real forecasts accumulate actual loads in the `Forecast` table, the task should compare against those instead. At the time of the supplied database the `Forecast` table is empty.

9. Serving Layer, Backend and Dashboard

9.1 Production model wrapper (inference/production_model.py)

`ProductionModel` reads `registry.json` and exposes one interface, `predict()`, whether the production entry is the naive baseline or a saved candidate. It is the only place that inspects the type of the production entry. For a baseline, the required input is just `lag_24` and the prediction is that value. For a candidate, the class loads the artifact, takes the feature list from its metadata and predicts from exactly those columns. It raises an error for an unknown type or for a candidate entry without an artifact folder. The method `info()` returns the type, name, MAE and promotion time, and `required_features` tells callers which inputs are needed.

9.2 FastAPI ML service (main.py, routes/predict.py, schemas/predict.py)

The service is created with a lifespan handler that loads the production model at start-up and releases it at shutdown. CORS is enabled for the Vite development origins. Table 18 lists the endpoints.

Table 18: Endpoints of the FastAPI ML service (port 8000)

Method and path	Purpose
GET /health	Liveness and the model currently serving ("starting" until the model is loaded)
GET /model	Name, type, MAE, promotion time and required features of the production model; 503 if not loaded
GET /metrics	Recorded metrics of the production entry (the baseline has MAE only)
POST /predict	Day-ahead forecast for one target hour; cached; 422 if required features are missing; 503 if no model
POST /predict/batch	List of requests; each item succeeds or fails independently, with an error message

The request schema `PredictRequest` has a mandatory `timestamp` (validated by `pd.Timestamp`) and optional feature fields. Every feature is optional **in the schema** because the API contract must not change when the production model changes: what is actually required is decided at request time against `ProductionModel.required_features`. Two fields have aliases that match the real column names with spaces (`specific humidity` and `wind speed`). `to_feature_row()` builds a one-row `DataFrame` indexed by the target timestamp from the fields that were set. The response contains the `timestamp`, `predicted_load_mw`, `model_type` and `model_name`.

9.3 Prediction cache (caching/prediction_cache.py)

Repeated identical requests, for example from a polling dashboard, should not recompute. The cache key is the SHA-256 hash of the request payload serialised as JSON with sorted keys, so it does not depend on the order of fields, and it carries the prefix `gridpredict:prediction:.` Entries expire after 300 seconds. The cache is designed to **fail open**: a Redis error on read returns a miss, and an error on write is swallowed and logged as a warning, so a cache outage can never break prediction serving. The module comment records a practical lesson from the project's own environment, where the host name `localhost` failed to resolve while `127.0.0.1` worked; the front end and the Redis configuration therefore use `127.0.0.1` explicitly. The client has a two-second connection timeout and is created lazily on first use rather than at import time.

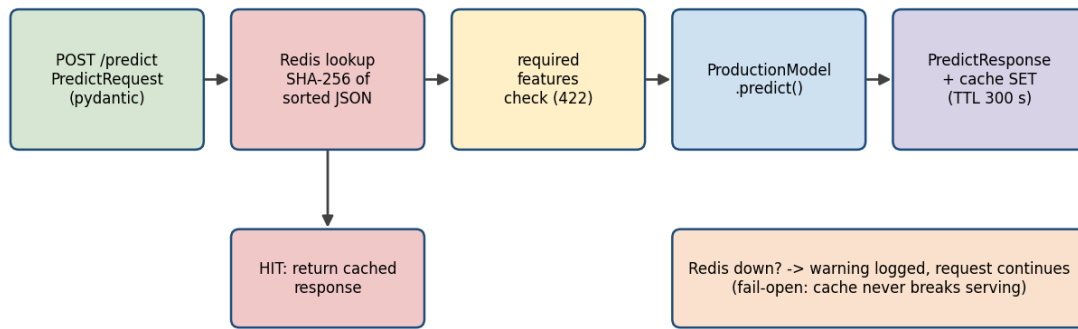


Figure 24: Life cycle of a POST /predict request, including the cache hit path and the fail-open behaviour.

9.4 Django REST backend

The Django project config contains four apps: `forecasts`, `model_registry`, `monitoring` and `users`. The database is SQLite (path from the `SQLITE_PATH` environment variable), the only DRF renderer is JSON, and CORS allows the Vite origins. Three models mirror the ML pipeline: `Forecast` (target timestamp, predicted load, actual load that is filled in later, model type and name), `ModelVersion` (one promotion decision with its metrics and reason) and `DataMonitoring` (one drift and performance check). The viewsets are read-only because forecasts, promotion decisions and monitoring records are created by the ML pipeline and Celery, never by API clients.

Table 19: Endpoints of the Django backend (port 8001)

Path	Content
/api/forecasts/	Stored forecasts (empty in the supplied database)
/api/model-versions/	Production model and rejected challengers (3 rows in the supplied database)
/api/monitoring/	Drift and performance checks (1 row)
/api/historical-demand/	Daily minimum, mean and maximum load from the processed dataset
/api/feature-importance/	Importances from the latest saved random_forest artifact
/api/modeling-findings/	Raw text of docs/MODELING_FINDINGS.md for display with react-markdown
/health/redis/ and /	Redis ping (503 on failure) and a basic status message

The management command `sync_registry` copies the real `models/registry.json` into the `ModelVersion` table: it deletes the existing rows, writes the current production entry as `is_current_production = True`, and then writes one row per history entry with status "production" or "rejected". If the baseline was never promoted, it uses the time of the first history entry as the evaluation time.

9.5 React dashboard

The dashboard is a React 19 single-page application built with Vite and styled with Tailwind CSS; it uses React Router for navigation, Axios for the API clients, Recharts for charts and react-markdown for the findings page. It has eight pages (Figure 25). Two Axios clients are defined, one for the Django API and one for the FastAPI service, both with a five-second timeout.

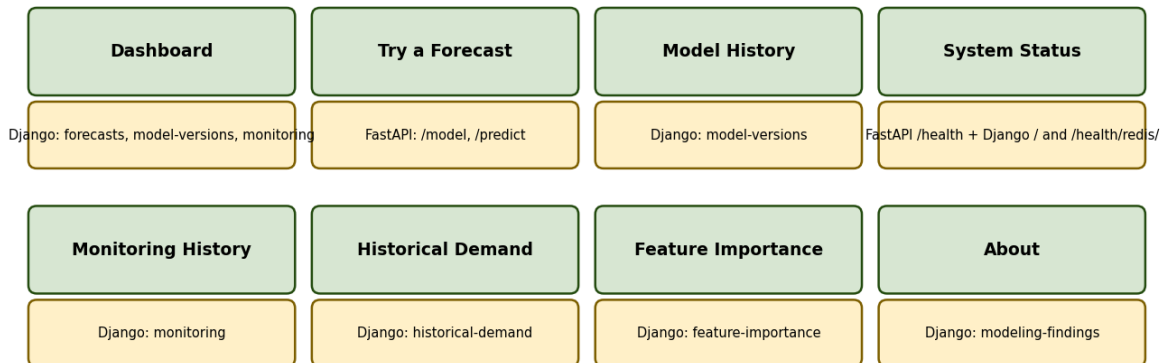


Figure 25: The eight pages of the dashboard and the back-end endpoints that feed each of them.

9.6 Containerised deployment

`docker-compose.yml` defines six services (Table 20). The FastAPI image copies only the ML package, data and models; the Django image also copies the backend and docs and runs `python manage.py migrate` before starting the server; the Celery worker and beat services reuse the Django image with different commands; and the front end is built in a Node 22 stage and served by nginx, with a rule that returns `index.html` for any route so that React Router paths work on refresh. The Django data volume `django_data` holds the SQLite file and the Celery beat schedule.

Table 20: Services in `docker-compose.yml`

Service	Port	Role
redis (redis:7-alpine)	6379	Celery broker and prediction cache
fastapi	8000	ML prediction service
django	8001	REST API and database migrations
celery_worker	none	Executes monitoring tasks (celery -A config worker)
celery_beat	none	Schedules the daily check (celery -A config beat)
frontend	5173 → 80	nginx serving the built React application

A security note: `settings.py` contains an insecure development fallback for `SECRET_KEY` and defaults `DEBUG` to `True`. These are acceptable for local development but must be replaced by environment-provided values before any public deployment.

10. Testing and Quality Assurance

The project contains **187 test functions**: 165 in the `tests` directory for the ML package and 22 in the Django apps. The numbers in this chapter are counts taken from the source files; the pass/fail output of a test run should be attached from the command in Appendix F. The tests are organised one module per source file, and they test behaviour rather than implementation details.

Table 21: Test functions by area

Area	Test modules	Functions
Preprocessing	data_loader, calendar_features, lag_features, rolling_features, pipeline	33
Splitting and training	split, baseline, model_factory, trainer, compare, train_pipeline, train_candidate, artifacts	48
Evaluation and backtesting	metrics, selection, backtest_selection, backtest	39
Registry and monitoring	registry, drift, performance_and_retraining	26
Serving	fastapi_health, fastapi_model_metrics, fastapi_predict, fastapi_predict_batch, prediction_cache	19
Django backend	forecasts, model_registry, sync_registry, monitoring, monitoring tasks	22
Total		187

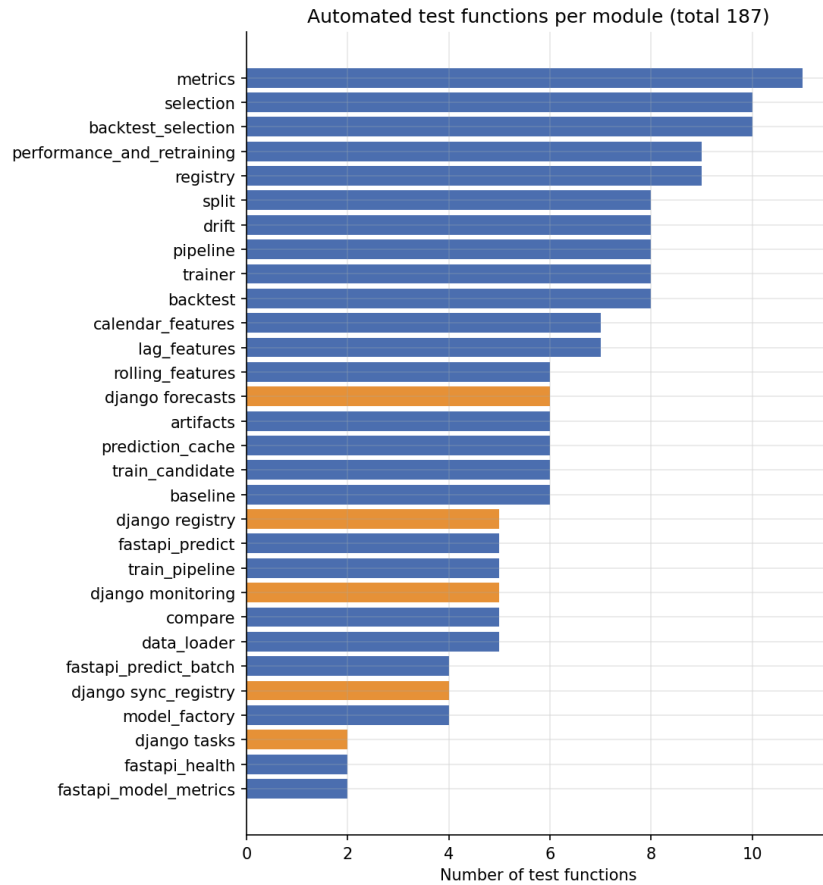


Figure 26: Number of test functions per module (blue: ML package, orange: Django apps).

10.1 Tests that protect the most important guarantees

- **No leakage.** `test_features_ignore_load_from_t_minus_23h_to_t` overwrites the load from $t - 23\text{ h}$ to t with 9999 and checks that the features at t are unchanged. Other tests check that lags use timestamps and not row positions, that lags shorter than the horizon are rejected (`test_lag_shorter_than_horizon_is_rejected`), that rolling features ignore hours after the forecast time and that they match a manual calculation.
- **Honest evaluation.** `test_the_test_period_is_never_used` and `test_training_stops_at_least_24h_before_each_fold` protect the backtest. The split tests check that the parts add up to the whole, are in time order without overlap, and that unsorted input or wrong boundaries are rejected.
- **Selection and promotion.** The selection and backtest-selection tests cover eligibility, tie-breaking and the "no winner" case. The registry tests include `test_our_real_result_would_not_promote_either_candidate`, which encodes the outcome of Chapter 7 as a regression test.
- **Monitoring.** Drift tests check identical distributions (no drift), all columns shifted (drift), one shifted column out of three (no dataset drift), a configurable threshold and `test_real_train_vs_test_split_shows_major_drift`. Performance tests

check errors just inside and just above the 15 % threshold, and the retraining tests check that drift alone, degradation alone and both together produce the right reason.

- **Serving.** The cache tests check that the key does not depend on field order, that Redis errors are swallowed on read and write, and one test round-trips through a real Redis server. The API tests cover health, metrics, single and batch prediction.

The test file names and the complete list of test functions are in Appendix H.

11. Discussion

11.1 What the results mean

The central result is negative but valuable: **no trained model was good enough to replace "same hour yesterday" on the held-out period.** This does not mean that machine learning is unsuitable for this problem. It means that, with one year of data, an untuned tree ensemble does not follow a rising load level as well as a rule that copies the most recent observation. The error analysis sharpens the statement: the Random Forest is slightly better than the baseline in hours with load up to 100 MW but much worse in the high-load hours, which are the hours that matter most to a grid operator.

11.2 The value of honest evaluation

The project shows twice how a model that looks good can fail. XGBoost won on validation and failed on test. The Random Forest won the more careful backtest and failed on test as well. Because the selection rule was fixed before the comparison, the test set was used only for reporting, and the promotion gate compared each challenger with the production model on shared rows, the failure was detected before deployment instead of after. This is the behaviour recommended by the literature on rolling-origin evaluation (Tashman, 2000) and on the governance of machine learning systems (Sculley et al., 2015).

11.3 Strengths

- A complete life cycle is implemented, not only a model: data, features, selection, registry, promotion, monitoring, serving, scheduling and dashboard.
- Leakage is prevented at three levels (configuration assertion, function validation, end-to-end tests).
- Failure is handled explicitly: invalid metric input raises errors, an unknown drift method raises an error, and Redis failure does not stop predictions.
- Decisions are recorded: model metadata, registry history and the monitoring table keep an audit trail.
- The system is reproducible: fixed random seed, pinned dependencies, containers and a migration-defined schedule.

11.4 Limitations and threats to validity

- **One year of data.** Seasonal effects cannot be separated from a trend, and the test period contains a season that the training data have not seen. The conclusion that the level shift causes the failure is a hypothesis supported by evidence, not a proven cause.
- **Weather at the target hour is the actual value**, not a forecast. Real accuracy would likely be lower for all models, including candidates.
- **The test set was looked at twice:** once for XGBoost and once for the Random Forest candidate. The Random Forest result is therefore a report and not a clean evaluation (stated in its metadata).
- **Untuned models.** The settings are defaults. Tuning on the validation set only, or on backtest folds, might change the ranking.
- **One city, one target.** Results may not generalise to other grids.
- **The monitoring job uses a static split**, a hard-coded reference MAE of 9.01 MW, and no live forecasts (the `Forecast` table is empty).
- **Seasonal drift triggers an alert.** With a reference period that is not seasonally matched and an OR rule, a normal seasonal change can recommend retraining while the production model's error is unchanged (Section 8.5.1).
- **Recomputations in this report.** The backtest folds, test error analysis and drift distances were recomputed with different library versions from the pinned ones, and Evidently was not available, so those figures are consistent with, but not identical to, the project outputs.
- **Development security defaults** (insecure fallback secret key, DEBUG on by default, permissive hosts in some settings) must be hardened for production.

11.5 Recommendations

(i) Collect at least two or three years of hourly data so that all seasons appear in training and several seasons can be held out. (ii) Make the model level-aware, for example by predicting the ratio or the difference to the lag-24 or a slow seasonal baseline (the `delta_baseline` mode of the backtest is a first step), or by blending the model output with the baseline. (iii) Replace actual weather by archived weather forecasts for training and evaluation. (iv) Use a seasonally matched reference window for drift (for example the same months of the previous year) and consider requiring evidence of both drift and degradation before retraining. (v) Store live forecasts and actual loads so that monitoring uses real production errors.

12. Conclusions and Future Work

GridPredict meets its objectives. It cleans and documents the data without inventing values, builds leakage-safe calendar, weather, lag and rolling features, compares three models against two naive baselines, selects candidates with rules that were fixed in advance, reports the held-out result honestly, protects the production model with a promotion gate, monitors drift and performance with Evidently and a combined retraining rule, and serves predictions through cached FastAPI endpoints, a Django REST backend, a daily Celery job and a React dashboard in six containers.

The quantitative outcome is that the naive "same hour yesterday" rule (test MAE 9.01 MW) remains in production, because the XGBoost model (10.67 MW) and the backtest-selected Random Forest (10.40 MW) were both worse on the held-out period and were rejected by the gate. The pipeline therefore did what it was designed to do. Future work should focus on more data, level-aware modelling, forecast-based weather inputs, probabilistic forecasts and live monitoring with real production errors.

References

- Bergmeir, C. and Benítez, J.M. (2012) 'On the use of cross-validation for time series predictor evaluation', *Information Sciences*, 191, pp. 192–213.
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32.
- Chai, T. and Draxler, R.R. (2014) 'Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature', *Geoscientific Model Development*, 7(3), pp. 1247–1250.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: a scalable tree boosting system', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. San Francisco, CA, 13–17 August. New York: ACM, pp. 785–794.
- Docker Inc. (n.d.) *Docker documentation*. Available at: <https://docs.docker.com/> (Accessed: 3 October 2026).
- Django Software Foundation (n.d.) *Django documentation*. Available at: <https://docs.djangoproject.com/> (Accessed: 3 October 2026).
- Evidently AI (n.d.) *Evidently documentation: data drift*. Available at: <https://docs.evidentlyai.com/> (Accessed: 3 October 2026).
- Friedman, J.H. (2001) 'Greedy function approximation: a gradient boosting machine', *The Annals of Statistics*, 29(5), pp. 1189–1232.
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. and Bouchachia, A. (2014) 'A survey on concept drift adaptation', *ACM Computing Surveys*, 46(4), article 44.
- Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020) 'Array programming with NumPy', *Nature*, 585(7825), pp. 357–362.

- Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The elements of statistical learning: data mining, inference, and prediction*. 2nd edn. New York: Springer.
- Hippert, H.S., Pedreira, C.E. and Souza, R.C. (2001) 'Neural networks for short-term load forecasting: a review and evaluation', *IEEE Transactions on Power Systems*, 16(1), pp. 44–55.
- Hong, T. and Fan, S. (2016) 'Probabilistic electric load forecasting: a tutorial review', *International Journal of Forecasting*, 32(3), pp. 914–938.
- Hunter, J.D. (2007) 'Matplotlib: a 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90–95.
- Hyndman, R.J. and Athanasopoulos, G. (2021) *Forecasting: principles and practice*. 3rd edn. Melbourne: OTexts. Available at: <https://otexts.com/fpp3/> (Accessed: 3 October 2026).
- Hyndman, R.J. and Koehler, A.B. (2006) 'Another look at measures of forecast accuracy', *International Journal of Forecasting*, 22(4), pp. 679–688.
- Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. (2012) 'Leakage in data mining: formulation, detection, and avoidance', *ACM Transactions on Knowledge Discovery from Data*, 6(4), article 15.
- Ke, G., Meng, Q., Finley, T. et al. (2017) 'LightGBM: a highly efficient gradient boosting decision tree', in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Red Hook, NY: Curran Associates, pp. 3146–3154.
- Lin, J. (1991) 'Divergence measures based on the Shannon entropy', *IEEE Transactions on Information Theory*, 37(1), pp. 145–151.
- McKinney, W. (2010) 'Data structures for statistical computing in Python', in *Proceedings of the 9th Python in Science Conference*, pp. 56–61.
- Meta Open Source (n.d.) *React documentation*. Available at: <https://react.dev/> (Accessed: 3 October 2026).
- Pedregosa, F., Varoquaux, G., Gramfort, A. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Pydantic (n.d.) *Pydantic documentation*. Available at: <https://docs.pydantic.dev/> (Accessed: 3 October 2026).
- Ramírez, S. (n.d.) *FastAPI documentation*. Available at: <https://fastapi.tiangolo.com/> (Accessed: 3 October 2026).
- Redis Ltd (n.d.) *Redis documentation*. Available at: <https://redis.io/docs/> (Accessed: 3 October 2026).
- Sculley, D., Holt, G., Golovin, D. et al. (2015) 'Hidden technical debt in machine learning systems', in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*. Red Hook, NY: Curran Associates, pp. 2503–2511.
- Tashman, L.J. (2000) 'Out-of-sample tests of forecasting accuracy: an analysis and review', *International Journal of Forecasting*, 16(4), pp. 437–450.

- Vallender, S.S. (1974) 'Calculation of the Wasserstein distance between probability distributions on the line', *Theory of Probability and Its Applications*, 18(4), pp. 784–786.
- Virtanen, P., Gommers, R., Oliphant, T.E. et al. (2020) 'SciPy 1.0: fundamental algorithms for scientific computing in Python', *Nature Methods*, 17(3), pp. 261–272.
- Willmott, C.J. and Matsuura, K. (2005) 'Advantages of the mean absolute error (MAE) over the root mean squared error (RMSE) in assessing average model performance', *Climate Research*, 30(1), pp. 79–82.
- Project data and code: the Ahmedabad hourly weather and electric load dataset (2021–2022), file "weather and load dataset.csv", and the GridPredict source code, repository ParasRimal/GridDemand-Forecasting-AI, GitHub (Accessed: 3 October 2026).

Appendix A. Source File Inventory

The table below lists the Python modules of the ML package with their length in lines.

Table 22: Modules of ml_service/app

File	Lines	Purpose
config.py	151	Central configuration: paths, cleaning, horizon, lags, split, models, thresholds
main.py	74	FastAPI app, lifespan loading, /health /model /metrics
preprocessing/data_loader.py	90	Load, validate, timeline check, clean
preprocessing/calendar_features.py	40	Seven calendar features incl. Gujarat holidays
preprocessing/lag_features.py	59	Leakage-safe lag features by timestamp
preprocessing/rolling_features.py	69	Rolling mean/std ending at $t - 24$ h
preprocessing/pipeline.py	68	Feature table assembly, warm-up removal, save/load
training/split.py	59	Chronological train/validation/test split
training/baseline.py	34	Naive lag baselines
training/model_factory.py	39	Build RF, XGBoost, LightGBM by name
training/trainer.py	46	Train, predict, score
training/compare.py	43	Compare models on validation and select
training/backtest.py	89	Expanding-window monthly backtest
training/train_pipeline.py	119	Full training run and test report
training/train_candidate.py	98	Train the backtest-selected RF candidate
training/artifacts.py	70	Save/load model + metadata
training/registry.py	121	registry.json and promotion gate
evaluation/metrics.py	75	MAE, RMSE, MAPE, R^2
evaluation/selection.py	66	Validation selection rule
evaluation/backtest_selection.py	89	Backtest selection rule
monitoring/drift.py	124	Evidently drift wrapper
monitoring/performance.py	53	MAE degradation check
monitoring/retraining.py	68	Combined retraining decision

File	Lines	Purpose
inference/production_model.py	73	Registry-driven prediction wrapper
caching/prediction_cache.py	69	Redis cache, fail-open
routes/predict.py	89	POST /predict and /predict/batch
schemas/predict.py	81	Pydantic request/response schemas

Table 23: Inspection scripts in the notebooks folder

Script	Purpose
inspect_powerload.py	Shape, types and summary of the PowerLoad dataset
inspect_signal.py	Correlations and autocorrelation of the PowerLoad dataset (evidence for rejecting it)
inspect_gaps.py	Gaps between consecutive timestamps in the PowerLoad dataset
inspect_ahmedabad.py	Shape, column names and summary of the Ahmedabad dataset
inspect_signal_ahmedabad.py	Daily, weekly and monthly profiles, correlations and autocorrelation of the Ahmedabad data
inspect_anomalies.py	The -999 irradiance block and the load dropout runs in the Ahmedabad data
compare_variants.py	Rolling backtest of four feature/target variants for the three models

Appendix B. Central Configuration (config.py)

The complete configuration file, which holds every threshold, window, parameter and path of the pipeline.

config.py (ml_service/app/config.py)

```
"""Central configuration for the GridPredict ML pipeline.

Every decision made during data inspection lives here, so no other file
needs to hardcode these values.
"""
from pathlib import Path
import os

# ----- paths
# This file is ml_service/app/config.py -> project root is 2 levels up.
PROJECT_ROOT = Path(__file__).resolve().parents[2]
RAW_DATA_PATH = PROJECT_ROOT / "data" / "raw" / "weather and load dataset.csv"
PROCESSED_DATA_PATH = PROJECT_ROOT / "data" / "processed" / "ahmedabad_processed.csv"

# ----- dataset
# Primary dataset: Ahmedabad hourly weather + electric load (Nov 2021 - Nov 2022).
# PowerLoad_Dataset.csv was rejected as primary: its load is statistically
# indistinguishable from random noise (see notebooks/inspect_signal.py).
TARGET_COLUMN = "Electric Load (MW)"
DATE_PARTS = ["YEAR", "Month", "Day", "Hour"]
WEATHER_COLUMNS = ["irradiance", "temperature", "dewpoint", "specific humidity", "wind speed"]

# ----- cleaning
# Both rules turn bad values into NaN. Raw CSV is never edited, and target
# values are never invented by interpolation.
IRRADIANCE_MISSING_CODE = -999.0 # sensor "no measurement" code (24 hours on 2022-01-07/08)
MIN_VALID_LOAD_MW = 0.0 # load <= this is treated as a meter dropout (79 rows)

# ----- forecast
# Day-ahead forecasting: predict the load 24 hours after the moment of forecast.
FORECAST_HORIZON_HOURS = 24
LAG_HOURS = [24, 48, 168] # 1 day, 2 days, 1 week

# Leakage guard: a lag shorter than the horizon would use the future.
assert all(lag >= FORECAST_HORIZON_HOURS for lag in LAG_HOURS), (
    "Every lag must be >= FORECAST_HORIZON_HOURS, otherwise the future leaks into the features."
)

# ----- chronological split
# Boundaries are inclusive. No shuffling, and the test set is the latest period.
TRAIN_END = "2022-06-30 23:00:00"
VALIDATION_END = "2022-08-31 23:00:00"
# Test = everything after VALIDATION_END.

# ----- holidays
# Ahmedabad is in Gujarat, India. Holidays are known in advance, so no leakage.
HOLIDAY_COUNTRY = "IN"
HOLIDAY_SUBDIVISION = "GJ"

# ----- rolling
# Windows END at (target hour - horizon), so they only use loads known at forecast time.
ROLLING_WINDOWS_HOURS = [24, 168] # last day, last week
ROLLING_MIN_FRACTION = 0.5 # a window needs >= 50% of its hours to have data

# ----- weather
# Decision: use the ACTUAL weather at the target hour. In production this stands
# in for a weather forecast, so real-world accuracy would be somewhat lower.
# Phase 2 compares models with, without, and with lagged weather.
WEATHER_ASSUMPTION = "actual_at_target_hour"

# ----- training
# Fixed seed so training is reproducible. Model settings below are sensible
# defaults, NOT tuned. Any later tuning must use the validation set only.
RANDOM_SEED = 42
RANDOM_FOREST_PARAMS = {
    "n_estimators": 300, # number of trees
    "min_samples_leaf": 2, # each leaf needs >= 2 rows, which reduces memorizing noise
    "max_features": 0.5, # each split looks at half the features, so trees differ
    "n_jobs": -1, # use all CPU cores
}

XGBOOST_PARAMS = {
    "n_estimators": 300,
    "learning_rate": 0.05, # small steps, so each tree corrects only part of the error
}
```

```

    "max_depth": 6,
    "subsample": 0.8,          # each tree sees 80% of the rows
    "colsample_bytree": 0.8,   # and 80% of the features
    "tree_method": "hist",
    "n_jobs": -1,
}
LIGHTGBM_PARAMS = {
    "n_estimators": 300,
    "learning_rate": 0.05,
    "num_leaves": 31,
    "min_child_samples": 20,   # each leaf needs >= 20 rows
    "subsample": 0.8,
    "subsample_freq": 1,      # LightGBM only applies `subsample` if this is >= 1
    "colsample_bytree": 0.8,
    "deterministic": True,    # same data + same seed = same model
    "n_jobs": -1,
    "verbose": -1,
}

# ----- selection
# Rule: a model is eligible only if its validation MAE is strictly below the baseline's.
# The lowest MAE wins. Eligible models within the tolerance of the best MAE are
# treated as tied, and the lowest RMSE among them wins. The test set is never used.
BASELINE_LAG_HOURS = 24      # "same hour yesterday"
SELECTION_TIE_TOLERANCE_MW = 0.1

# ----- artifacts
# Trained models are saved here, one timestamped folder per save.
MODELS_DIR = PROJECT_ROOT / "models"

# ----- backtest-based selection
# A candidate is eligible only if it beats the baseline in at least this share of
# backtest folds (0.7 -> 5 of 7 folds) AND its mean MAE is below the baseline's.
# Among eligible candidates within the tolerance of the best mean MAE, the one that
# beats the baseline in the most folds wins. The test set is never used.
BACKTEST_MIN_FOLD_FRACTION = 0.7
BACKTEST_TIE_TOLERANCE_MW = 0.1
# Calendar columns that only identify the season (the "no_season" variant drops them).
SEASON_FEATURES = ["month", "quarter", "week_of_year"]

# ----- promotion gate
# A challenger is promoted only if it beats the CURRENT production model's
# MAE by more than this margin, on the evaluation rows both are scored on.
# This avoids swapping the production model over noise-sized differences.
PROMOTION_MIN_MAE_IMPROVEMENT_MW = 0.3
REGISTRY_PATH = MODELS_DIR / "registry.json"

# ----- drift
# Evidently's DriftedColumnsCount flags a column as drifted using its own
# per-method thresholds (Wasserstein distance normed for numeric columns,
# Jensen-Shannon distance for categorical/binary columns, both at 0.1 by
# default). DRIFT_SHARE_THRESHOLD is OUR threshold on top of that: the
# fraction of features that must be individually flagged before we consider
# the DATASET AS A WHOLE to have drifted meaningfully. This deliberately
# avoids "retrain whenever data changes" -- a couple of flaky columns should
# not trigger anything.
DRIFT_SHARE_THRESHOLD = 0.5

# ----- retraining decision
# Performance is "degraded" when recent MAE exceeds the reference MAE by
# more than this fraction (0.15 = 15% worse). A fraction (not a fixed MW
# margin) is used because MAE naturally varies with the season's load level.
PERFORMANCE_DEGRADATION_THRESHOLD = 0.15
# Retraining is triggered if drift OR degradation is present (per project
# requirements: "statistically significant drift AND/OR performance
# degradation" -- not on data changing alone).

# ----- caching
# Predictions are cached in Redis, keyed by their exact input values, so
# repeated identical requests (e.g. dashboard polling) don't recompute.
# If Redis is unreachable, caching fails open: predictions still work, just
# without caching (logged as a warning), so a cache outage never breaks
# the actual service.
REDIS_HOST = os.environ.get("REDIS_HOST", "127.0.0.1")
REDIS_PORT = int(os.environ.get("REDIS_PORT", "6379"))
PREDICTION_CACHE_TTL_SECONDS = 300
PREDICTION_CACHE_KEY_PREFIX = "gridpredict:prediction:"

```

Appendix C. Core Source Code Listings

The listings below are the unmodified source of the modules that implement the main algorithms of the project.

C.1 data_loader.py (ml_service/app/preprocessing/data_loader.py)

```

"""Load, validate and clean the Ahmedabad weather + load dataset.

Cleaning rules (all defined in config.py):
* irradiance == -999 -> NaN (sensor "no measurement" code)
* load <= 0 -> NaN (meter dropout)
Bad values become NaN. We never interpolate the target.
"""
from __future__ import annotations

import logging
from pathlib import Path

import pandas as pd

from ml_service.app import config

logger = logging.getLogger(__name__)

REQUIRED_COLUMNS = config.DATE_PARTS + config.WEATHER_COLUMNS + [config.TARGET_COLUMN]

def load_raw(path: Path = config.RAW_DATA_PATH) -> pd.DataFrame:
    """Read the raw CSV and check that every required column is present."""
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f"Raw data file not found: {path}")

    df = pd.read_csv(path)
    missing_cols = [c for c in REQUIRED_COLUMNS if c not in df.columns]
    if missing_cols:
        raise ValueError(f"Missing required columns: {missing_cols}")

    logger.info("Loaded %d rows from %s", len(df), path.name)
    return df

def build_timestamp_index(df: pd.DataFrame) -> pd.DataFrame:
    """Replace YEAR/Month/Day/Hour with a single DatetimeIndex called 'Timestamp'."""
    timestamps = pd.to_datetime(
        dict(year=df["YEAR"], month=df["Month"], day=df["Day"], hour=df["Hour"])
    )
    out = df[config.WEATHER_COLUMNS + [config.TARGET_COLUMN]].copy()
    out.index = pd.DatetimeIndex(timestamps, name="Timestamp")
    return out

def check_timeline(df: pd.DataFrame) -> pd.DataFrame:
    """Make the index unique, sorted and strictly hourly."""
    dupes = int(df.index.duplicated().sum())
    if dupes > 0:
        raise ValueError(f"Found {dupes} duplicate timestamps; refusing to continue.")

    if not df.index.is_monotonic_increasing:
        logger.warning("Timestamps were not sorted; sorting chronologically.")
        df = df.sort_index()

    full_index = pd.date_range(df.index.min(), df.index.max(), freq="h", name="Timestamp")
    n_missing = len(full_index) - len(df)
    if n_missing > 0:
        logger.warning("%d hourly timestamps were missing; adding them as NaN rows.", n_missing)
        df = df.reindex(full_index)
    else:
        logger.info("Timeline check passed: %d rows, every hour present.", len(df))
    return df

def clean(df: pd.DataFrame) -> pd.DataFrame:
    """Turn known-bad values into NaN (no filling, no deleting rows)."""
    df = df.copy()

    bad_irradiance = df["irradiance"] == config.IRRADIANCE_MISSING_CODE
    df.loc[bad_irradiance, "irradiance"] = float("nan")

    bad_load = df[config.TARGET_COLUMN] <= config.MIN_VALID_LOAD_MW

```

```

df.loc[bad_load, config.TARGET_COLUMN] = float("nan")

logger.info(
    "Cleaning: %d irradiance values and %d load values set to NaN.",
    int(bad_irradiance.sum()),
    int(bad_load.sum()),
)
return df

def load_clean_data(path: Path = config.RAW_DATA_PATH) -> pd.DataFrame:
    """Full loading pipeline: read -> timestamp -> timeline check -> clean."""
    df = load_raw(path)
    df = build_timestamp_index(df)
    df = check_timeline(df)
    return clean(df)

```

C.2 lag_features.py (ml_service/app/preprocessing/lag_features.py)

```

"""Lag features for the day-ahead forecast.

For a row at target hour t, lag_h is the load at exactly t - h hours.
Because every h >= FORECAST_HORIZON_HOURS (24), the value was already known
at forecast time, so the future cannot leak into the features.

Lags are looked up by TIMESTAMP, not by row position, so they stay correct
even if a row is missing. Unknown values stay NaN (no interpolation).
"""
from __future__ import annotations

import logging
from typing import Optional, Sequence

import pandas as pd

from ml_service.app import config

logger = logging.getLogger(__name__)

def lag_column_name(hours: int) -> str:
    """Name of the lag column, e.g. 24 -> 'lag_24'."""
    return f"lag_{hours}"

def add_lag_features(
    df: pd.DataFrame,
    lags: Optional[Sequence[int]] = None,
    target: str = config.TARGET_COLUMN,
) -> pd.DataFrame:
    """Return a copy of df with one lag column per entry in `lags`."""
    if lags is None:
        lags = config.LAG_HOURS

    if not isinstance(df.index, pd.DatetimeIndex):
        raise TypeError("df must have a DatetimeIndex.")
    if not df.index.is_unique:
        raise ValueError("df index has duplicate timestamps.")
    if target not in df.columns:
        raise KeyError(f"Target column '{target}' not found.")

    too_short = [lag for lag in lags if lag < config.FORECAST_HORIZON_HOURS]
    if too_short:
        raise ValueError(
            f"Lags {too_short} are shorter than the {config.FORECAST_HORIZON_HOURS}h "
            "forecast horizon and would leak the future."
        )

    out = df.copy()
    series = df[target]
    for lag in lags:
        # shift(freq=...) moves each value forward in TIME by `lag` hours,
        # so the value from t - lag ends up at t. reindex aligns it to our rows.
        shifted = series.shift(freq=pd.Timedelta(hours=lag))
        out[lag_column_name(lag)] = shifted.reindex(out.index)

    logger.info("Added lag features: %s", [lag_column_name(lag) for lag in lags])
    return out

```

C.3 rolling_features.py (ml_service/app/preprocessing/rolling_features.py)

```

"""Rolling features for the day-ahead forecast.

For a row at target hour t, the forecast is made at t - 24 h, and the newest
load known at that moment is the load at t - 24. So every rolling window must
END at t - 24, never at t:

    rolling_mean_24 at t = mean of the load from t-47h to t-24h    (24 hours)
    rolling_mean_168 at t = mean of the load from t-191h to t-24h  (168 hours)

Method: compute an ordinary trailing rolling statistic that ends at each hour s,
then move the result forward in TIME by 24 h so it lands on t = s + 24.
Missing loads are ignored inside a window. A window needs at least
ROLLING_MIN_FRACTION of its hours to have data, otherwise the result is NaN.
"""
from __future__ import annotations

import logging
import math
from typing import Optional, Sequence

import pandas as pd

from ml_service.app import config

logger = logging.getLogger(__name__)

def rolling_column_names(window: int) -> tuple[str, str]:
    """Names of the mean and std columns, e.g. 24 -> ('rolling_mean_24', 'rolling_std_24')."""
    return f"rolling_mean_{window}", f"rolling_std_{window}"

def add_rolling_features(
    df: pd.DataFrame,
    windows: Optional[Sequence[int]] = None,
    target: str = config.TARGET_COLUMN,
) -> pd.DataFrame:
    """Return a copy of df with rolling mean/std columns that end at t - horizon."""
    if windows is None:
        windows = config.ROLLING_WINDOWS_HOURS

    if not isinstance(df.index, pd.DatetimeIndex):
        raise TypeError("df must have a DatetimeIndex.")
    if not df.index.is_unique:
        raise ValueError("df index has duplicate timestamps.")
    if not df.index.is_monotonic_increasing:
        raise ValueError("df index must be sorted oldest to newest.")
    if target not in df.columns:
        raise KeyError(f"Target column '{target}' not found.")
    too_small = [w for w in windows if w < 2]
    if too_small:
        raise ValueError(f"Windows {too_small} are too small; a std needs at least 2 hours.")

    out = df.copy()
    series = df[target]
    horizon = pd.Timedelta(hours=config.FORECAST_HORIZON_HOURS)

    for w in windows:
        min_periods = max(2, math.ceil(w * config.ROLLING_MIN_FRACTION))
        # Trailing window over the last w hours, including hour s itself.
        roll = series.rolling(f"{w}h", min_periods=min_periods, closed="right")
        mean_col, std_col = rolling_column_names(w)
        # shift(freq=..) moves each result forward in TIME by the horizon,
        # so the window that ended at s = t - 24 lands on row t.
        out[mean_col] = roll.mean().shift(freq=horizon).reindex(out.index)
        out[std_col] = roll.std().shift(freq=horizon).reindex(out.index)

    logger.info("Added rolling features for windows %s", list(windows))
    return out

```

C.4 pipeline.py (ml_service/app/preprocessing/pipeline.py)

```

"""Full feature pipeline: raw CSV -> model-ready table.

Steps: load + clean -> calendar -> lags -> rolling -> keep usable rows.
Row filtering happens LAST, so lags and rolling windows are computed on the
complete hourly timeline and stay exact.
"""
from __future__ import annotations

```

```

import logging
from pathlib import Path
from typing import Optional

import pandas as pd

from ml_service.app import config
from ml_service.app.preprocessing.calendar_features import CALENDAR_FEATURES,
add_calendar_features
from ml_service.app.preprocessing.data_loader import load_clean_data
from ml_service.app.preprocessing.lag_features import add_lag_features, lag_column_name
from ml_service.app.preprocessing.rolling_features import add_rolling_features,
rolling_column_names

logger = logging.getLogger(__name__)

def feature_columns() -> list[str]:
    """The model input columns, in a fixed order (the target is NOT included)."""
    cols = list(CALENDAR_FEATURES) + list(config.WEATHER_COLUMNS)
    cols += [lag_column_name(lag) for lag in config.LAG_HOURS]
    for window in config.ROLLING_WINDOWS_HOURS:
        cols += list(rolling_column_names(window))
    return cols

def build_features(clean: pd.DataFrame) -> pd.DataFrame:
    """Turn the cleaned hourly table into the model-ready feature table."""
    df = add_calendar_features(clean)
    df = add_lag_features(df)
    df = add_rolling_features(df)
    df = df[feature_columns() + [config.TARGET_COLUMN]]

    n_start = len(df)
    warmup_end = df.index.min() + pd.Timedelta(hours=max(config.LAG_HOURS))
    df = df[df.index >= warmup_end]
    n_after_warmup = len(df)
    df = df[df[config.TARGET_COLUMN].notna()]

    logger.info(
        "Rows: %d -> %d after dropping the %dh warm-up -> %d after dropping missing targets.",
        n_start, n_after_warmup, max(config.LAG_HOURS), len(df),
    )
    return df

def build_and_save(output_path: Optional[Path] = None) -> pd.DataFrame:
    """Run the whole pipeline from the raw CSV and save the result as CSV."""
    output_path = Path(output_path or config.PROCESSED_DATA_PATH)
    df = build_features(load_clean_data())
    output_path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(output_path)
    logger.info("Saved %d rows x %d columns to %s", len(df), df.shape[1], output_path)
    return df

def load_processed(path: Optional[Path] = None) -> pd.DataFrame:
    """Read the saved processed dataset back, with its Timestamp index."""
    path = Path(path or config.PROCESSED_DATA_PATH)
    if not path.exists():
        raise FileNotFoundError(f"Processed data not found: {path}. Run build_and_save() first.")
    return pd.read_csv(path, index_col="Timestamp", parse_dates=["Timestamp"])

```

C.5 backtest.py (ml_service/app/training/backtest.py)

```

"""Rolling-origin (expanding window) backtest, using train + validation data ONLY.

For each calendar month M from `first_fold` on:
    train on all rows before (start of M - gap), then test on month M.
The gap is the forecast horizon (24 h): labels from the last 24 hours before M
would not be known yet when the forecast for M's first hour is made.

The test period (after config.VALIDATION_END) is never used.
Each fold scores the model and the lag baseline on the SAME rows.

target_mode:
    "level"          the model predicts the load directly.
    "delta_baseline" the model predicts (load - baseline lag) and the lag is
                    added back. Training rows with an unknown lag are skipped.
"""
from __future__ import annotations

```

```

import logging
from typing import Optional, Sequence

import pandas as pd

from ml_service.app import config
from ml_service.app.evaluation.metrics import mae
from ml_service.app.preprocessing.pipeline import feature_columns
from ml_service.app.training.model_factory import build_model

logger = logging.getLogger(__name__)

TARGET_MODES = ("level", "delta_baseline")

def backtest(
    df: pd.DataFrame,
    model_name: str,
    first_fold: str = "2022-02",
    features: Optional[Sequence[str]] = None,
    gap_hours: Optional[int] = None,
    target_mode: str = "level",
) -> pd.DataFrame:
    """Return one row per monthly fold with model and baseline scores."""
    if target_mode not in TARGET_MODES:
        raise ValueError(f"target_mode must be one of {TARGET_MODES}, got '{target_mode}'.")
    features = list(features) if features is not None else feature_columns()
    gap = pd.Timedelta(hours=config.FORECAST_HORIZON_HOURS if gap_hours is None else gap_hours)
    baseline_col = f"lag_{config.BASELINE_LAG_HOURS}"
    target = config.TARGET_COLUMN
    delta = target_mode == "delta_baseline"

    data = df[df.index <= pd.Timestamp(config.VALIDATION_END)] # never touch the test period
    if data.empty:
        raise ValueError("No data available before the end of validation.")

    rows = []
    for month in pd.period_range(first_fold, data.index.max().to_period("M"), freq="M"):
        start, end = month.start_time, (month + 1).start_time
        train_part = data[data.index < start - gap]
        if delta:
            train_part = train_part[train_part[baseline_col].notna()]
        test_part = data[(data.index >= start) & (data.index < end)]
        test_part = test_part[test_part[baseline_col].notna()] # rows the baseline can predict
        if train_part.empty or test_part.empty:
            continue

        y_train = train_part[target] - train_part[baseline_col] if delta else train_part[target]
        model = build_model(model_name)
        model.fit(train_part[features], y_train)
        raw = model.predict(test_part[features])
        if delta:
            raw = raw + test_part[baseline_col].to_numpy()
        pred = pd.Series(raw, index=test_part.index)
        y = test_part[target]

        model_mae = mae(y, pred)
        baseline_mae = mae(y, test_part[baseline_col])
        rows.append({
            "month": str(month),
            "train_rows": len(train_part),
            "test_rows": len(test_part),
            "actual_mean": float(y.mean()),
            "model_bias": float((pred - y).mean()),
            "model_mae": model_mae,
            "baseline_mae": baseline_mae,
            "beats_baseline": model_mae < baseline_mae,
        })

    if not rows:
        raise ValueError("No usable folds; check first_fold against the data range.")
    return pd.DataFrame(rows).set_index("month")

```

C.6 selection.py (ml_service/app/evaluation/selection.py)

```

"""Automatic model selection on validation scores.

Rule (documented in config.py):
1. A model is eligible only if its MAE is strictly below the baseline MAE.
2. Among eligible models, the lowest MAE wins.
3. Eligible models within `tie_tolerance_mw` of the best MAE are tied,
   and the lowest RMSE among them wins.

```

```

4. If no model is eligible, there is no winner (winner is None).
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Mapping, Optional

import pandas as pd

from ml_service.app import config

_EPSILON = 1e-12 # guards the tolerance comparison against floating-point noise

@dataclass(frozen=True)
class SelectionResult:
    winner: Optional[str]
    eligible: list[str]
    reason: str
    table: pd.DataFrame # one row per model, sorted by MAE

def select_best_model(
    scores: Mapping[str, Mapping[str, float]],
    baseline_mae: float,
    tie_tolerance_mw: float = config.SELECTION_TIE_TOLERANCE_MW,
) -> SelectionResult:
    """Apply the selection rule to {model_name: {'mae': ..., 'rmse': ..., ...}}."""
    if not scores:
        raise ValueError("No model scores were given.")
    for name, s in scores.items():
        if "mae" not in s or "rmse" not in s:
            raise ValueError(f"Scores for '{name}' must include 'mae' and 'rmse'.")

    table = pd.DataFrame({name: dict(s) for name, s in scores.items()}).T
    table["eligible"] = table["mae"] < baseline_mae
    table = table.sort_values("mae")

    eligible = [n for n in table.index if table.loc[n, "eligible"]]
    if not eligible:
        return SelectionResult(
            winner=None,
            eligible=[],
            reason=f"No model beat the baseline MAE of {baseline_mae:.2f} MW.",
            table=table,
        )

    best_mae = min(scores[n]["mae"] for n in eligible)
    contenders = [n for n in eligible if scores[n]["mae"] <= best_mae + tie_tolerance_mw +
        _EPSILON]
    winner = min(contenders, key=lambda n: (scores[n]["rmse"], scores[n]["mae"], n))

    if len(contenders) == 1:
        reason = f"{winner} has the lowest validation MAE ({scores[winner]['mae']:.2f} MW)."
    else:
        reason = (
            f"{'', '.join(sorted(contenders))} are within {tie_tolerance_mw} MW of the best MAE "
            f"({best_mae:.2f} MW); {winner} has the lowest RMSE ({scores[winner]['rmse']:.2f}
MW)."
        )

    return SelectionResult(winner=winner, eligible=eligible, reason=reason, table=table)

```

C.7 backtest_selection.py (ml_service/app/evaluation/backtest_selection.py)

```

"""Model selection from rolling-backtest results (many time windows, not one).

Input: {candidate_name: DataFrame returned by training.backtest.backtest()}.

Rule (settings in config.py):
1. Eligible = beats the baseline in >= BACKTEST_MIN_FOLD_FRACTION of the folds
   AND mean model MAE < mean baseline MAE.
2. The lowest mean MAE among eligible candidates wins.
3. Eligible candidates within the tolerance of the best mean MAE are tied;
   the one that beats the baseline in the most folds wins.
4. If nothing is eligible, winner is None.
"""
from __future__ import annotations

import math
from typing import Mapping, Optional

import pandas as pd

```

```

from ml_service.app import config
from ml_service.app.evaluation.selection import SelectionResult

REQUIRED_COLUMNS = ["model_mae", "baseline_mae"]
_EPSILON = 1e-12 # guards the tolerance comparison against floating-point noise

def select_from_backtests(
    results: Mapping[str, pd.DataFrame],
    min_fold_fraction: float = config.BACKTEST_MIN_FOLD_FRACTION,
    tie_tolerance_mw: float = config.BACKTEST_TIE_TOLERANCE_MW,
) -> SelectionResult:
    """Apply the selection rule to a dict of backtest result tables."""
    if not results:
        raise ValueError("No backtest results were given.")

    rows = []
    for name, bt in results.items():
        missing = [c for c in REQUIRED_COLUMNS if c not in bt.columns]
        if missing:
            raise ValueError(f"Backtest for '{name}' is missing columns: {missing}")
        if bt.empty:
            raise ValueError(f"Backtest for '{name}' has no folds.")

        n_folds = len(bt)
        # round() first: 10 * 0.7 is 7.000000000000001 in floating point, and ceil would give 8.
        folds_needed = math.ceil(round(n_folds * min_fold_fraction, 9))
        folds_beaten = int((bt["model_mae"] < bt["baseline_mae"]).sum())
        mean_mae = float(bt["model_mae"].mean())
        baseline_mean_mae = float(bt["baseline_mae"].mean())
        rows.append({
            "name": name,
            "folds": n_folds,
            "folds_beaten": folds_beaten,
            "folds_needed": folds_needed,
            "mean_mae": mean_mae,
            "baseline_mean_mae": baseline_mean_mae,
            "eligible": folds_beaten >= folds_needed and mean_mae < baseline_mean_mae,
        })

    table = pd.DataFrame(rows).set_index("name").sort_values("mean_mae")
    eligible = table[table["eligible"]]
    if eligible.empty:
        return SelectionResult(
            winner=None,
            eligible=[],
            reason="No candidate beat the baseline in enough backtest folds with a lower mean
MAE.",
            table=table,
        )

    best = float(eligible["mean_mae"].min())
    contenders = eligible[eligible["mean_mae"] <= best + tie_tolerance_mw + _EPSILON]
    ranked = contenders.reset_index().sort_values(
        ["folds_beaten", "mean_mae", "name"], ascending=[False, True, True]
    )
    winner = str(ranked.iloc[0]["name"])
    w = table.loc[winner]

    if len(contenders) == 1:
        reason = (
            f"{winner} has the lowest mean backtest MAE ({w['mean_mae']:.2f} MW vs baseline "
            f"{w['baseline_mean_mae']:.2f}) and beats the baseline in "
            f"{int(w['folds_beaten'])}/{int(w['folds'])} folds."
        )
    else:
        reason = (
            f"{'', ' '.join(sorted(contenders.index))} are within {tie_tolerance_mw} MW of the best
mean MAE "
            f"({best:.2f} MW); {winner} beats the baseline in the most folds "
            f"({int(w['folds_beaten'])}/{int(w['folds'])})."
        )
    return SelectionResult(winner=winner, eligible=list(eligible.index), reason=reason,
table=table)

```

C.8 registry.py (ml_service/app/training/registry.py)

```

"""Production model registry: what is currently in production, and why.

registry.json (tracked in Git, unlike model files) records:
- production: the model/rule currently serving predictions

```

```

- history: every promotion decision ever made, promoted or not

Default production, before anything is ever promoted, is the naive
"same hour yesterday" baseline. A challenger only replaces production
if it beats it by more than PROMOTION_MIN_MAE_IMPROVEMENT_MW on a fair,
shared evaluation.
"""
from __future__ import annotations

import json
import logging
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Optional

from ml_service.app import config

logger = logging.getLogger(__name__)

BASELINE_ENTRY = {
    "type": "baseline",
    "name": "naive_lag_24",
    "description": "Same hour yesterday (lag_24). Default production model until a challenger
proves better.",
}

def _default_registry() -> dict:
    return {"production": dict(BASELINE_ENTRY), "history": []}

def load_registry(path: Optional[Path] = None) -> dict:
    """Load registry.json, or return the baseline-default registry if it doesn't exist yet."""
    path = Path(path or config.REGISTRY_PATH)
    if not path.exists():
        return _default_registry()
    with open(path, encoding="utf-8") as f:
        return json.load(f)

def save_registry(registry: dict, path: Optional[Path] = None) -> Path:
    path = Path(path or config.REGISTRY_PATH)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(registry, f, indent=2, default=str)
    return path

def evaluate_promotion(
    challenger_name: str,
    challenger_mae: float,
    challenger_metrics: dict,
    production_mae: float,
    min_improvement_mw: float = config.PROMOTION_MIN_MAE_IMPROVEMENT_MW,
) -> dict:
    """Decide whether a challenger should replace the current production model.

    Returns a decision record; does NOT mutate the registry (see promote()).
    """
    improvement = production_mae - challenger_mae
    promoted = improvement > min_improvement_mw
    return {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "challenger": challenger_name,
        "challenger_mae": challenger_mae,
        "challenger_metrics": dict(challenger_metrics),
        "production_mae_before": production_mae,
        "improvement_mw": improvement,
        "min_improvement_required_mw": min_improvement_mw,
        "promoted": promoted,
        "reason": (
            f"{challenger_name} beat production by {improvement:.2f} MW "
            f"> required {min_improvement_mw} MW)."
            if promoted else
            f"{challenger_name} did not beat production by more than {min_improvement_mw} MW "
            f"(improvement was {improvement:.2f} MW)."
        ),
    }

def promote(
    challenger_entry: dict[str, Any],
    challenger_mae: float,
    challenger_metrics: dict,
    path: Optional[Path] = None,

```

```

    min_improvement_mw: float = config.PROMOTION_MIN_MAE_IMPROVEMENT_MW,
) -> dict:
    """Run the promotion decision against the CURRENT registry, apply it if it wins,
    record it in history either way, and save. Returns the decision record.
    """
    registry = load_registry(path)
    production_mae = registry["production"].get("mae")
    if production_mae is None:
        raise ValueError(
            "Current production entry has no recorded 'mae'; cannot compare fairly. "
            "Re-score production on the same evaluation rows as the challenger first."
        )

    decision = evaluate_promotion(
        challenger_entry.get("name", "candidate"),
        challenger_mae,
        challenger_metrics,
        production_mae,
        min_improvement_mw,
    )
    registry["history"].append(decision)
    if decision["promoted"]:
        new_production = dict(challenger_entry)
        new_production["mae"] = challenger_mae
        new_production["metrics"] = dict(challenger_metrics)
        new_production["promoted_at"] = decision["timestamp"]
        registry["production"] = new_production
        logger.info("PROMOTED: %s", decision["reason"])
    else:
        logger.info("NOT promoted: %s", decision["reason"])

    save_registry(registry, path)
    return decision

```

C.9 drift.py (ml_service/app/monitoring/drift.py)

```

"""Feature drift detection using Evidently (0.7.x API).

Wraps evidently.Report + DataDriftPreset so the rest of the codebase works
with a plain dict, not Evidently's internal Snapshot/JSON structure.

Evidently auto-selects a drift-detection method per column depending on its
type, and different method families point in OPPOSITE directions:
- distance methods (Wasserstein, Jensen-Shannon, PSI, ...): drifted when
  value > threshold (a bigger distance means more different).
- statistical-test p-value methods (K-S, Z-test, chi-square, ...): drifted
  when value < threshold (a small p-value rejects "same distribution").
Evidently's own Snapshot.dict()["tests"] list is empty for this preset in
this version, so we cannot rely on it for a ready-made pass/fail; we
determine "drifted" ourselves per method family below.

DRIFT_SHARE_THRESHOLD is OUR threshold on top of the per-column flags: the
fraction of columns that must be individually flagged before the dataset as
a whole is considered meaningfully drifted. This deliberately avoids
"retrain whenever data changes" -- a couple of flaky columns should not
trigger anything.
"""
from __future__ import annotations

from typing import Optional, Sequence

import pandas as pd
from evidently import Dataset, DataDefinition, Report
from evidently.preset import DataDriftPreset

from ml_service.app import config

# Method name (as reported in config["method"]) -> comparison direction.
# "above": drifted when value > threshold (distance-style methods).
# "below": drifted when value < threshold (p-value-style methods).
_METHOD_DIRECTION = {
    "Wasserstein distance (normed)": "above",
    "Jensen-Shannon distance": "above",
    "Population Stability Index": "above",
    "K-S p_value": "below",
    "Z-test p_value": "below",
    "Chi-square p_value": "below",
}

def _is_drifted(method: str, value: float, threshold: float) -> bool:
    direction = _METHOD_DIRECTION.get(method)

```

```

    if direction is None:
        raise ValueError(
            f"Unknown drift method '{method}'; add its comparison direction to "
            f"_METHOD_DIRECTION in drift.py before trusting this result."
        )
    return bool(value > threshold) if direction == "above" else bool(value < threshold)

def _extract(snapshot_dict: dict) -> dict:
    """Turn Evidently's raw metrics list into {'summary': ..., 'columns': {...}}.

    The summary's own drifted-count/share (from DriftedColumnsCount) is NOT
    used here, because we recompute "drifted" per column ourselves (see
    module docstring on p-value direction), so our count may legitimately
    differ from Evidently's if its internal test wiring disagrees.
    """
    columns = {}
    for entry in snapshot_dict["metrics"]:
        cfg = entry.get("config", {})
        if not cfg.get("type", "").endswith("ValueDrift"):
            continue
        column, method, threshold, value = cfg["column"], cfg.get("method"),
        cfg.get("threshold"), entry["value"]
        columns[column] = {
            "method": method,
            "threshold": threshold,
            "drift_score": value,
            "drifted": _is_drifted(method, value, threshold),
        }
    if not columns:
        raise ValueError("No ValueDrift entries found in the Evidently report.")
    return {"columns": columns}

def check_drift(
    reference: pd.DataFrame,
    current: pd.DataFrame,
    columns: Optional[Sequence[str]] = None,
    drift_share_threshold: float = config.DRIFT_SHARE_THRESHOLD,
) -> dict:
    """Compare `current` against `reference` and return a drift summary.

    Returns:
    {
        "dataset_drifted": bool,          # share of drifted columns > threshold
        "drifted_column_share": float,
        "drifted_column_count": int,
        "n_columns": int,
        "threshold_used": float,
        "columns": {col: {"method", "threshold", "drift_score", "drifted"}, ...},
    }
    """
    cols = list(columns) if columns is not None else list(reference.columns)
    missing_ref = [c for c in cols if c not in reference.columns]
    missing_cur = [c for c in cols if c not in current.columns]
    if missing_ref or missing_cur:
        raise KeyError(f"Columns missing from reference={missing_ref}, current={missing_cur}")
    if len(reference) == 0 or len(current) == 0:
        raise ValueError("reference and current must both be non-empty.")

    ref_ds = Dataset.from_pandas(reference[cols], data_definition=DataDefinition())
    cur_ds = Dataset.from_pandas(current[cols], data_definition=DataDefinition())

    report = Report([DataDriftPreset()])
    snapshot = report.run(cur_ds, ref_ds)
    extracted = _extract(snapshot.dict())

    n = len(extracted["columns"])
    n_drifted = sum(1 for c in extracted["columns"].values() if c["drifted"])
    share = n_drifted / n

    return {
        "dataset_drifted": bool(share > drift_share_threshold),
        "drifted_column_share": share,
        "drifted_column_count": n_drifted,
        "n_columns": n,
        "threshold_used": drift_share_threshold,
        "columns": extracted["columns"],
    }

```

C.10 retraining.py (ml_service/app/monitoring/retraining.py)

```

"""Combined retraining decision: drift and/or performance degradation.

Per project requirement: retraining must be based on STATISTICALLY MEANINGFUL
evidence (drift and/or degradation), never "retrain whenever data changes."
Both checks below already have their own documented thresholds
(config.DRIFT_SHARE_THRESHOLD, config.PERFORMANCE_DEGRADATION_THRESHOLD);
this module just combines their verdicts and logs a clear reason, mirroring
the promotion gate's style (evaluation.selection / training.registry).
"""
from __future__ import annotations

from typing import Optional, Sequence

import pandas as pd

from ml_service.app.monitoring.drift import check_drift
from ml_service.app.monitoring.performance import check_performance

def decide_retraining(
    reference_features: pd.DataFrame,
    current_features: pd.DataFrame,
    y_true: pd.Series,
    y_pred: pd.Series,
    reference_mae: float,
    feature_columns: Optional[Sequence[str]] = None,
) -> dict:
    """Run both checks and return a single retraining decision.

    Returns:
        {
            "retrain": bool,          # drift OR degradation
            "drift": {...},          # full check_drift() result
            "performance": {...},    # full check_performance() result
            "reason": str,
        }
    """
    drift_result = check_drift(reference_features, current_features, columns=feature_columns)
    perf_result = check_performance(y_true, y_pred, reference_mae)

    drifted = drift_result["dataset_drifted"]
    degraded = perf_result["degraded"]
    retrain = drifted or degraded

    parts = []
    if drifted:
        parts.append(
            f"data drift detected ({drift_result['drifted_column_share']:.0%} of "
            f"{drift_result['n_columns']} features drifted, threshold "
            f"{drift_result['threshold_used']:.0%})"
        )
    if degraded:
        parts.append(
            f"performance degraded (MAE {perf_result['recent_mae']:.2f} vs reference "
            f"{perf_result['reference_mae']:.2f}, +{perf_result['relative_increase']:.0%}, "
            f"threshold +{perf_result['threshold_used']:.0%})"
        )
    if retrain:
        reason = "Retraining recommended: " + "; ".join(parts) + "."
    else:
        reason = (
            f"No retraining needed: drift share {drift_result['drifted_column_share']:.0%} "
            f"(threshold {drift_result['threshold_used']:.0%}) and performance within "
            f"{perf_result['relative_increase']:.0%} of reference "
            f"(threshold +{perf_result['threshold_used']:.0%})."
        )

    return {"retrain": retrain, "drift": drift_result, "performance": perf_result, "reason": reason}

```

Appendix D. Model Metadata

The `metadata.json` files saved with the two candidate models (the field `model_params` is omitted for brevity).

D.1 `random_forest_20260925T014616395014Z`

```
{
  "status": "candidate",
  "selected_by": "rolling_backtest",
  "target": "Electric Load (MW)",
  "feature_columns": [
    "hour",
    "day_of_week",
    "is_weekend",
    "is_holiday",
    "irradiance",
    "temperature",
    "dewpoint",
    "specific humidity",
    "wind speed",
    "lag_24",
    "lag_48",
    "lag_168",
    "rolling_mean_24",
    "rolling_std_24",
    "rolling_mean_168",
    "rolling_std_168"
  ],
  "season_features_excluded": [
    "month",
    "quarter",
    "week_of_year"
  ],
  "forecast_horizon_hours": 24,
  "random_seed": 42,
  "trained_on": "train + validation (everything up to end of validation)",
  "train_val_rows": 7051,
  "train_val_start": "2021-11-08 05:00:00",
  "train_val_end": "2022-08-31 23:00:00",
  "test_start": "2022-09-01 00:00:00",
  "test_end": "2022-11-01 23:00:00",
  "test_scores": {
    "baseline": {
      "mae": 9.011933649932157,
      "rmse": 12.542486576108377,
      "mape": 11.834657202419994,
      "r2": 0.7127264468485083,
      "rows_scored": 1474,
      "rows_skipped": 7
    },
    "model_same_rows": {
      "mae": 10.401531594062222,
      "rmse": 13.237517956986286,
      "mape": 13.25046283124776,
      "r2": 0.6800062611951576
    },
    "model_all_rows": {
      "mae": 10.554687462585147,
      "rmse": 13.730900402035713,
      "mape": 13.316917723634033,
      "r2": 0.6647099994761301
    }
  },
  "test_score_note": "This test score is a REPORT ONLY. The test period was already viewed once for an earlier candidate, so this is not a clean unseen evaluation. It was not used to pick this candidate or tune any setting.",
  "beats_baseline_on_test": false,
  "model_name": "random_forest",
  "saved_at": "2026-09-25T01:46:16.395014+00:00"
}
```

D.2 `xgboost_20260924T122415296167Z`

```
{
  "status": "candidate",
  "target": "Electric Load (MW)",
  "feature_columns": [
    "hour",
    "day_of_week",
    "is_weekend",
```

```

    "month",
    "quarter",
    "week_of_year",
    "is_holiday",
    "irradiance",
    "temperature",
    "dewpoint",
    "specific humidity",
    "wind speed",
    "lag_24",
    "lag_48",
    "lag_168",
    "rolling_mean_24",
    "rolling_std_24",
    "rolling_mean_168",
    "rolling_std_168"
  ],
  "forecast_horizon_hours": 24,
  "lag_hours": [
    24,
    48,
    168
  ],
  "rolling_windows_hours": [
    24,
    168
  ],
  "weather_assumption": "actual_at_target_hour",
  "random_seed": 42,
  "trained_on": "train split only",
  "split": {
    "train_start": "2021-11-08 05:00:00",
    "train_end": "2022-06-30 23:00:00",
    "validation_start": "2022-07-01 00:00:00",
    "validation_end": "2022-08-31 23:00:00",
    "test_start": "2022-09-01 00:00:00",
    "test_end": "2022-11-01 23:00:00",
    "train_rows": 5583,
    "validation_rows": 1468,
    "test_rows": 1481
  },
  "selection_reason": "random_forest, xgboost are within 0.1 MW of the best MAE (8.47 MW); xgboost has the lowest RMSE (11.46 MW).",
  "validation_scores": {
    "mae": 8.468004823707876,
    "rmse": 11.457738727037116,
    "mape": 12.393792027227098,
    "r2": 0.4202347952359393
  },
  "validation_all_models": {
    "random_forest": {
      "mae": 8.517529799827296,
      "rmse": 11.584875043691913,
      "mape": 12.64396369325746,
      "r2": 0.40729713674165235
    },
    "xgboost": {
      "mae": 8.468004823707876,
      "rmse": 11.457738727037116,
      "mape": 12.393792027227098,
      "r2": 0.4202347952359393
    },
    "lightgbm": {
      "mae": 8.614101830619596,
      "rmse": 11.666209383662583,
      "mape": 12.684829801492148,
      "r2": 0.39894550228243797
    }
  },
  "validation_baseline": {
    "mae": 9.251947444751382,
    "rmse": 13.290601179521596,
    "mape": 12.784091034603765,
    "r2": 0.21991204944282416,
    "rows_scored": 1448,
    "rows_skipped": 20
  },
  "test_scores": {
    "baseline": {
      "mae": 9.011933649932157,
      "rmse": 12.542486576108377,
      "mape": 11.834657202419994,
      "r2": 0.7127264468485083,
      "rows_scored": 1474,
      "rows_skipped": 7
    },
    "model_same_rows": {
      "mae": 10.665986756592625,
      "rmse": 13.41930512439626,
      "mape": 13.792882443855921,
      "r2": 0.6711571435596815
    }
  }
}

```

```
    },  
    "model_all_rows": {  
      "mae": 10.812593178218865,  
      "rmse": 13.888570333877698,  
      "mape": 13.852759022095468,  
      "r2": 0.656965616894446  
    }  
  },  
  "model_name": "xgboost",  
  "saved_at": "2026-09-24T12:24:15.296167+00:00"  
}
```

Appendix E. Model Registry (models/registry.json)

```
{
  "production": {
    "type": "baseline",
    "name": "naive_lag_24",
    "description": "Same hour yesterday (lag_24). Default production model until a challenger proves better.",
    "mae": 9.01
  },
  "history": [
    {
      "timestamp": "2026-09-25T01:50:50.134835+00:00",
      "challenger": "xgboost_full",
      "challenger_mae": 10.67,
      "challenger_metrics": {
        "mae": 10.67,
        "rmse": 13.42,
        "mape": 13.8,
        "r2": 0.671
      },
      "production_mae_before": 9.01,
      "improvement_mw": -1.6600000000000001,
      "min_improvement_required_mw": 0.3,
      "promoted": false,
      "reason": "xgboost_full did not beat production by more than 0.3 MW (improvement was -1.66 MW).",
    },
    {
      "timestamp": "2026-09-25T01:50:50.135203+00:00",
      "challenger": "random_forest_no_season",
      "challenger_mae": 10.4,
      "challenger_metrics": {
        "mae": 10.4,
        "rmse": 13.24,
        "mape": 13.3,
        "r2": 0.68
      },
      "production_mae_before": 9.01,
      "improvement_mw": -1.3900000000000006,
      "min_improvement_required_mw": 0.3,
      "promoted": false,
      "reason": "random_forest_no_season did not beat production by more than 0.3 MW (improvement was -1.39
MW).",
    }
  ]
}
```

Appendix F. Reproduction Instructions

All commands are run from the project root. The first block starts the complete system in containers; the second block lists the individual commands that follow from the source code.

```
# Whole system (redis, fastapi, django, celery_worker, celery_beat, frontend)
docker compose up --build
# Front end: http://localhost:5173   Django API: :8001/api/   ML API: :8000/docs
```

```
# Install the pinned dependencies
pip install -r requirements.txt

# Build the processed feature table
python -c "from ml_service.app.preprocessing.pipeline import build_and_save; build_and_save()"

# Train and compare models (validation selection, one test report)
python -m ml_service.app.training.train_pipeline

# Train the backtest-selected Random Forest candidate
python -m ml_service.app.training.train_candidate

# Compare feature and target variants by rolling backtest
python notebooks/compare_variants.py

# ML service
uvicorn ml_service.app.main:app --reload --port 8000

# Django backend, registry sync and Celery
cd backend
python manage.py migrate
python manage.py sync_registry
python manage.py runserver 8001
celery -A config worker --loglevel=info
celery -A config beat --loglevel=info

# Tests
pytest tests

# Front end
npm install && npm run dev
```

Appendix G. API Example (illustrative)

The following request and response are illustrations of the contract defined by the schemas. While the production model is the baseline, only `timestamp` and `lag_24` are required, and the baseline returns the lag value. The numbers are example values and are not recorded output of the system.

```
POST /predict
{
  "timestamp": "2022-09-15T14:00:00",
  "lag_24": 82.5
}

200 OK
{
  "timestamp": "2022-09-15T14:00:00",
  "predicted_load_mw": 82.5,
  "model_type": "baseline",
  "model_name": "naive_lag_24"
}
```

Appendix H. Complete Test Inventory

The table below lists every test module with its number of test functions and the names of the functions (without the prefix "test_").

Table 24: Test modules and functions

Module	Count	Test functions
test_artifacts	6	save_creates_both_files_in_a_new_folder; loaded_model_predicts_identically; metadata_round_trip_and_added_fields; numpy_numbers_in_metadata_are_saved_as_plain_numbers; input_metadata_is_not_modified; loading_a_missing_folder_is_rejected
test_backtest	8	the_test_period_is_never_used; training_stops_at_least_24h_before_each_fold; no_usable_folds_raises; output_has_one_row_per_month_and_expected_columns; beats_flag_matches_the_scores; unknown_target_mode_is_rejected; delta_mode_trains_on_the_change_from_the_lag; delta_mode_adds_the_lag_back_to_the_predictions
test_backtest_selection	10	lowest_mean_mae_wins_when_not_a_tie; tie_within_tolerance_the_candidate_beating_more_folds_wins; outside_tolerance_the_lower_mean_still_wins; beating_the_baseline_in_too_few_folds_is_not_eligible; enough_folds_but_a_worse_mean_is_not_eligible; no_eligible_candidate_returns_no_winner; fold_requirement_scales_with_the_number_of_folds; empty_input_is_rejected; missing_columns_are_rejected; table_lists_every_candidate_sorted_by_mean_mae
test_baseline	6	prediction_is_the_lag_column; rows_with_unknown_lag_are_skipped_and_counted; missing_lag_column_is_rejected; all_nan_lag_is_rejected; inputs_are_not_modified; real_validation_rows_are_all_accounted_for
test_calendar_features	7	all_seven_calendar_columns_are_added; no_missing_values_in_any_calendar_column; diwali_2021_is_correctly_flagged_as_a_holiday; a_known_sunday_is_flagged_as_weekend_not_holiday; weekend_flag_is_true_only_for_saturday_and_sunday; holiday_count_matches_the_known_27_gujarat_holidays; requires_a_datetime_index
test_compare	5	every_model_gets_scores_and_is_returned; baseline_and_models_are_scored_on_the_same_rows; winner_beats_the_baseline; unknown_model_is_rejected; missing_baseline_column_is_rejected
test_data_loader	5	duplicate_timestamps_are_rejected; unsorted_timestamps_are_sorted_not_rejected; missing_hours_are_filled_as_nan_rows; load_raw_rejects_missing_required_columns; load_raw_rejects_a_nonexistent_file
test_drift	8	identical_distributions_show_no_drift; all_columns_shifted_shows_drift; only_one_of_three_columns_shifted_does_not_trigger_dataset_drift; threshold_is_configurable; column_subset_is_respected; missing_column_is_rejected; empty_input_is_rejected; real_train_vs_test_split_shows_major_drift
test_fastapi_health	2	health_returns_ok_and_current_production_info; health_reflects_the_real_registry_state
test_fastapi_model_metrics	2	model_endpoint_describes_the_baseline_and_its_required_features; metrics_endpoint_returns_the_baseline_mae
test_fastapi_predict	5	predict_with_only_lag_24_succeeds_against_the_baseline; predict_missing_the_required_feature_returns_422; predict_ignores_irrelevant_extra_fields_gracefully; invalid_timestamp_format_is_rejected; repeated_identical_request_is_served_from_cache
test_fastapi_predict_batch	4	batch_with_all_valid_items_succeeds; batch_partial_failure_does_not_fail_the_whole_batch; batch_results_preserve_input_order_via_index; empty_batch_returns_empty_results
test_lag_features	7	lag_points_to_exact_hours_earlier; first_rows_are_nan_and_the_rest_are_filled; lag_uses_timestamps_not_row_positions; lag_shorter_than_horizon_is_rejected; input_dataframe_is_not_modified; missing_load_stays_missing_in_the_lag; real_data_lags_match_the_earlier_hours
test_metrics	11	hand_calculated_example; perfect_prediction; predicting_the_mean_gives_r2_zero; matches_scikit_learn_on_random_data; accepts_pandas_series_with_different_indexes; length_mismatch_is_rejected; nan_is_rejected; empty_input_is_rejected; mape_rejects_a_zero_actual; r2_rejects_constant_actual_values; evaluate_returns_the_four_metrics_in_order

Module	Count	Test functions
test_model_factory	4	all_three_models_are_available; trains_and_predicts_with_nan_features; same_seed_gives_identical_predictions; learns_a_pattern_on_unseen_rows
test_performance_and_retraining	9	error_within_threshold_is_not_degraded; error_just_above_threshold_is_degraded; error_below_reference_is_not_degraded; non_positive_reference_mae_is_rejected; empty_input_is_rejected; no_drift_no_degradation_means_no_retrain; drift_alone_triggers_retrain; degradation_alone_triggers_retrain; both_drift_and_degradation_are_both_named_in_the_reason
test_pipeline	8	output_columns_are_features_plus_target; target_is_never_a_feature; no_missing_target; warmup_rows_are_removed; index_is_sorted_and_unique; every_lag_column_respects_the_horizon; features_ignore_load_from_t_minus_23h_to_t; save_and_reload_round_trip
test_prediction_cache	6	cache_key_is_stable_regardless_of_dict_order; cache_key_differs_for_different_payloads; get_returns_none_when_redis_raises; set_does_not_raise_when_redis_raises; set_then_get_round_trips_through_real_redis; get_returns_none_for_a_key_never_set
test_registry	9	missing_registry_file_defaults_to_baseline_production; save_and_load_round_trip; clear_improvement_is_promoted; improvement_below_the_margin_is_not_promoted; a_worse_challenger_is_not_promoted; promote_updates_registry_when_challenger_wins; promote_keeps_production_when_challenger_loses_but_logs_the_attempt; promote_refuses_when_production_has_no_recorded_mae; our_real_result_would_not_promote_either_candidate
test_rolling_features	6	rolling_mean_and_std_match_the_formula; hours_after_forecast_time_cannot_change_the_feature; first_rows_are_nan_until_enough_history_exists; missing_loads_are_ignored_inside_the_window; input_dataframe_is_not_modified; real_data_matches_a_manual_calculation
test_selection	10	lowest_mae_wins_when_not_a_tie; tie_within_tolerance_lower_rmse_wins; just_outside_tolerance_does_not_use_rmse; model_that_loses_to_baseline_is_never_selected; equal_to_baseline_is_not_eligible; no_eligible_model_returns_no_winner; our_real_validation_scores_select_xgboost; empty_scores_are_rejected; scores_without_rmse_are_rejected; table_lists_every_model_sorted_by_mae
test_split	8	parts_add_up_to_the_whole_table; parts_are_in_time_order_without_overlap; boundaries_follow_the_config; expected_sizes_on_real_data; unsorted_input_is_rejected; empty_split_is_rejected; split_dates_in_wrong_order_are_rejected; split_xy_keeps_the_target_out_of_the_features
test_train_candidate	6	saved_in_exactly_one_folder; season_features_are_excluded; trained_on_train_plus_validation; metadata_is_explicit_about_the_test_score_caveat; test_scores_are_internally_consistent; saved_model_predicts_on_the_test_period
test_train_pipeline	5	a_winner_is_chosen_and_saved_in_exactly_one_folder; metadata_describes_the_model_honestly; metadata_has_validation_and_test_scores; saved_model_predicts_on_the_test_period; nothing_is_saved_when_no_model_beats_the_baseline
test_trainer	8	random_forest_is_available; unknown_model_is_rejected; prediction_keeps_the_timestamp_index; same_seed_gives_identical_predictions; model_learns_a_pattern_on_unseen_rows_despite_nan; missing_target_is_rejected; empty_training_data_is_rejected; score_model_only_uses_the_selected_rows
backend/forecasts/tests	6	forecast_can_be_created_without_an_actual_value_yet; actual_load_can_be_filled_in_later; default_ordering_is_newest_timestamp_first; list_endpoint_returns_created_forecasts; detail_endpoint_returns_one_forecast; write_methods_are_not_allowed
backend/model_registry/tests	5	rejected_candidate_can_be_recorded_with_only_mae; baseline_can_be_current_production; our_real_registry_history_can_be_recorded_as_two_rows; list_endpoint_returns_created_versions; write_methods_are_not_allowed
backend/model_registry/test_sync_registry	4	command_creates_expected_rows; command_is_idempotent_running_twice_does_not_duplicate; command_handles_a_missing_registry_file_gracefully; exactly_one_row_is_marked_current_production
backend/monitoring/tests	5	can_record_a_drift_only_retrain_decision; can_record_a_no_action_decision; default_ordering_is_newest_check_first; list_endpoint_returns_created_checks; write_methods_are_not_allowed
backend/monitoring/test_tasks	2	task_creates_a_data_monitoring_record; result_matches_our_known_real_world_finding

Appendix I. Environment and Pinned Versions

Table 25: Selected pinned dependencies (requirements.txt)

Package	Version	Package	Version
Django	6.0.8	scikit-learn	1.9.1
django-rest-framework	3.18.1	xgboost	3.4.1
celery	5.6.3	lightgbm	4.7.0
django-celery-beat	2.9.0	evidently	0.7.23
redis	8.1.0	pandas	3.0.6
fastapi	0.141.1	numpy	2.5.3
uvicorn	0.53.0	scipy	1.18.1
pydantic	2.13.5	holidays	0.105
pytest	9.1.1	joblib	1.6.0